

Optimization for Deep Learning

Choose the data · remember direction and scale · control the step

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

$$\theta_t \rightarrow \textbf{forward} \rightarrow \mathcal{L} \rightarrow \textbf{backward} \rightarrow g_t \rightarrow \textbf{optimizer} \rightarrow \theta_{t+1}$$

Backprop measured the slope; today we decide how to move

$\theta_t \rightarrow \text{forward} \rightarrow \mathcal{L} \rightarrow \text{backward} \rightarrow g_t \rightarrow \text{optimizer} \rightarrow \theta_{t+1}$

LECTURES 1–4

defined the loss, model, computation graph, and gradient g_t .

LECTURE 5

chooses the data, remembers useful history, and sets the step $\Delta\theta_t$.

$$\theta_t \rightarrow \textbf{forward} \rightarrow \mathcal{L} \rightarrow \textbf{backward} \rightarrow g_t \rightarrow \textbf{optimizer} \rightarrow \theta_{t+1}$$

LECTURES 1–4

defined the loss, model, computation graph, and gradient g_t .

LECTURE 5

chooses the data, remembers useful history, and sets the step $\Delta\theta_t$.

The gradient is a measurement. An optimizer turns it into $\Delta\theta_t := \theta_{t+1} - \theta_t$.

Outline

1. **Choose the data per update** — full batch, one example, or a minibatch
2. **Understand the estimate** — unbiased centre, variance, epochs, and updates
3. **Expose a geometric failure** — why an exact gradient can still zigzag and crawl
4. **Add useful memory** — EWMA → momentum → RMSProp → Adam
5. **Finish the training loop** — decay, PyTorch, and constrained parameters

Two examples will keep the mechanisms concrete

A • FOUR-POINT LINE FIT

$\hat{y} = b + wx$ on
 $(-1, -1), (0, 1), (1, 3), (2, 6)$.

Use: full/SGD/minibatch,
unbiasedness, variance, decay.

B • SCALED-FEATURE REGRESSION

Four two-feature examples with one
feature $10 \times$ larger.

Use: ravines, EWMA, momentum,
RMSProp, Adam.

Every contour, arrow, and trajectory is computed from a displayed objective. The notebooks reproduce the underlying calculations and seeded experiments.

**How much data should define
one update?**

What optimization means in this course

For scalar-output examples $(\mathbf{x}_i, y_i)$ and all trainable parameters $\boldsymbol{\theta}$,

$$\hat{y}_i = f_{\boldsymbol{\theta}}(\mathbf{x}_i), \quad \ell_{i(\boldsymbol{\theta})} = \ell(\hat{y}_i, y_i), \quad \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \ell_{i(\boldsymbol{\theta})}.$$

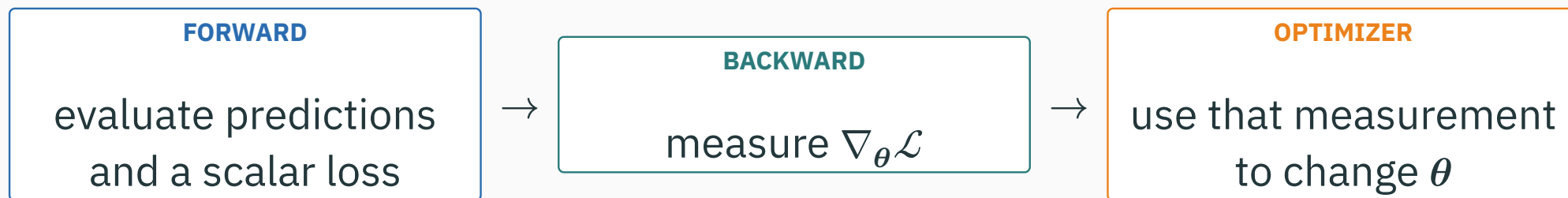
Course convention from L1–L3: bold lowercase/uppercase symbols are vectors/matrices; $y_i, \hat{y}_i, \ell_i$, and $\mathcal{L}$ are scalars.

What optimization means in this course

For scalar-output examples $(\mathbf{x}_i, y_i)$ and all trainable parameters $\boldsymbol{\theta}$,

$$\hat{y}_i = f_{\boldsymbol{\theta}}(\mathbf{x}_i), \quad \ell_{i(\boldsymbol{\theta})} = \ell(\hat{y}_i, y_i), \quad \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \ell_{i(\boldsymbol{\theta})}.$$

Course convention from L1–L3: bold lowercase/uppercase symbols are vectors/matrices; $y_i, \hat{y}_i, \ell_i$, and $\mathcal{L}$ are scalars.

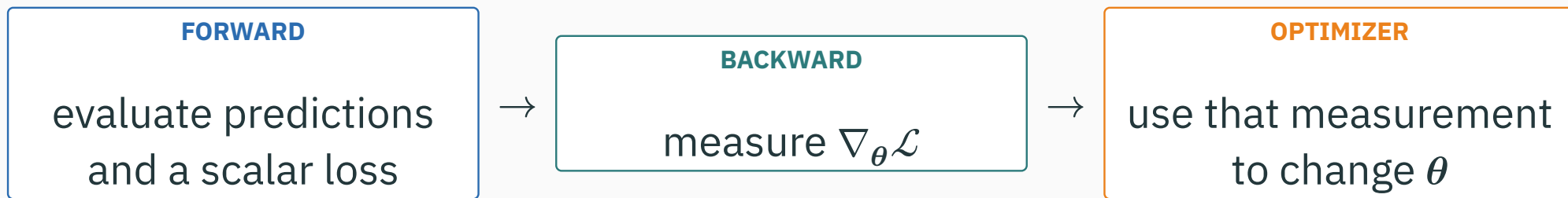


What optimization means in this course

For scalar-output examples $(\mathbf{x}_i, y_i)$ and all trainable parameters $\boldsymbol{\theta}$,

$$\hat{y}_i = f_{\boldsymbol{\theta}}(\mathbf{x}_i), \quad \ell_{i(\boldsymbol{\theta})} = \ell(\hat{y}_i, y_i), \quad \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \ell_{i(\boldsymbol{\theta})}.$$

Course convention from L1–L3: bold lowercase/uppercase symbols are vectors/matrices; $y_i, \hat{y}_i, \ell_i$, and $\mathcal{L}$ are scalars.



Today the objective stays fixed. We change which examples measure its gradient and how the optimizer transforms that measurement.

Revision: vanilla gradient descent is a four-line loop

For N examples and parameter vector θ ,

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_{i(\theta)}.$$

Revision: vanilla gradient descent is a four-line loop

For N examples and parameter vector θ ,

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_{i(\theta)}.$$

evaluate	$\mathcal{L}(\theta_t)$
differentiate	$g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$
choose η	$\eta > 0$
update	$\theta_{t+1} = \theta_t - \eta g_t$

Revision: vanilla gradient descent is a four-line loop

For N examples and parameter vector θ ,

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_{i(\theta)}.$$

evaluate	$\mathcal{L}(\theta_t)$
differentiate	$g_t = \nabla_{\theta} \mathcal{L}(\theta_t)$
choose η	$\eta > 0$
update	$\theta_{t+1} = \theta_t - \eta g_t$

Batch gradient descent (also called **full-batch GD**) uses all N examples for every update. “Vanilla GD” describes the update rule; the batch specifies which gradient it receives.

Example A: compute one full-batch gradient

Use $\hat{y}_i = b + wx_i$ and $\ell_i = \frac{1}{2}(\hat{y}_i - y_i)^2$. Then

$$\nabla_{\theta} \ell_i = (\hat{y}_i - y_i)(1, x_i).$$

Example A: compute one full-batch gradient

Use $\hat{y}_i = b + wx_i$ and $\ell_i = \frac{1}{2}(\hat{y}_i - y_i)^2$. Then

$$\nabla_{\boldsymbol{\theta}} \ell_i = (\hat{y}_i - y_i)(1, x_i).$$

At $\boldsymbol{\theta}_0 = (b, w) = (2, -1)$, the prediction becomes $\hat{y}_i = 2 - x_i$:

x_i	y_i	$\hat{y}_i = 2 - x_i$	$\hat{y}_i - y_i$	$\nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_0)$
-1	-1	$2 - (-1) = 3$	$3 - (-1) = 4$	$(4, -4)$
0	1	$2 - 0 = 2$	$2 - 1 = 1$	$(1, 0)$
1	3	$2 - 1 = 1$	$1 - 3 = -2$	$(-2, -2)$
2	6	$2 - 2 = 0$	$0 - 6 = -6$	$(-6, -12)$

Read each row left to right: predict $\hat{y}_i$, form the residual $\hat{y}_i - y_i$, then multiply by $(1, x_i)$ to obtain the gradient.

$$\mathbf{g}_0 = \mathbf{g}_{\text{full}} = \frac{1}{4}[(4, -4) + (1, 0) + (-2, -2) + (-6, -12)] = (-0.75, -4.5).$$

$$\mathbf{g}_0 = \mathbf{g}_{\text{full}} = \frac{1}{4}[(4, -4) + (1, 0) + (-2, -2) + (-6, -12)] = (-0.75, -4.5).$$

PARAMETER UPDATE

$$\boldsymbol{\theta}_1 = (2, -1) - 0.1(-0.75, -4.5) = (2.075, -0.55)$$

PRODUCED LINE

$$f_{\boldsymbol{\theta}_1}(x) = 2.075 - 0.55x$$

$$\mathbf{g}_0 = \mathbf{g}_{\text{full}} = \frac{1}{4}[(4, -4) + (1, 0) + (-2, -2) + (-6, -12)] = (-0.75, -4.5).$$

PARAMETER UPDATE

$$\boldsymbol{\theta}_1 = (2, -1) - 0.1(-0.75, -4.5) = (2.075, -0.55)$$

PRODUCED LINE

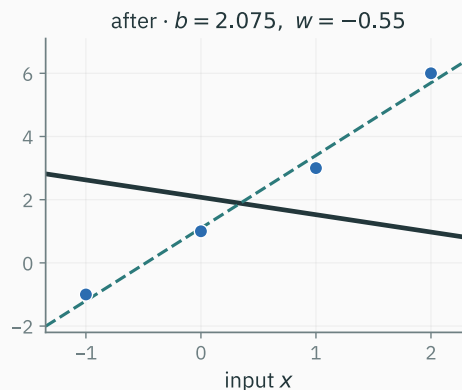
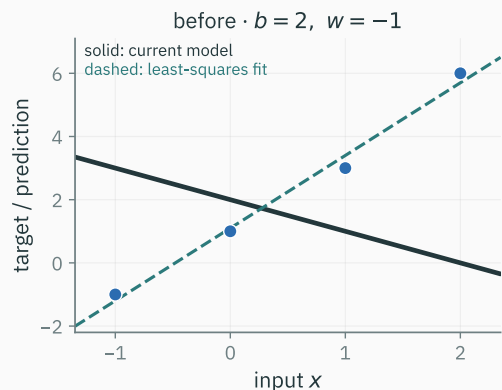
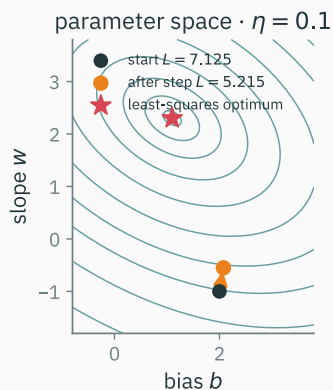
$$f_{\boldsymbol{\theta}_1}(x) = 2.075 - 0.55x$$

NEW RESIDUALS $e_{1,i} = f_{\boldsymbol{\theta}_1}(x_i) - y_i, \quad \mathbf{e}_1 = (3.625, 1.075, -1.475, -5.025)$

Here $e_{t,i}$ denotes residual error. Because $\ell_i = \frac{1}{2}e_i^2$ and $N = 4$, the full loss is $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{8}\|\mathbf{e}\|_2^2$.

before step	$\mathcal{L}(\boldsymbol{\theta}_0) = \frac{1}{8}(4^2 + 1^2 + (-2)^2 + (-6)^2) = 7.125$
after step	$\mathcal{L}(\boldsymbol{\theta}_1) = \frac{1}{8}(3.625^2 + 1.075^2 + (-1.475)^2 + (-5.025)^2) \approx 5.215$

One gradient step changes the fitted model



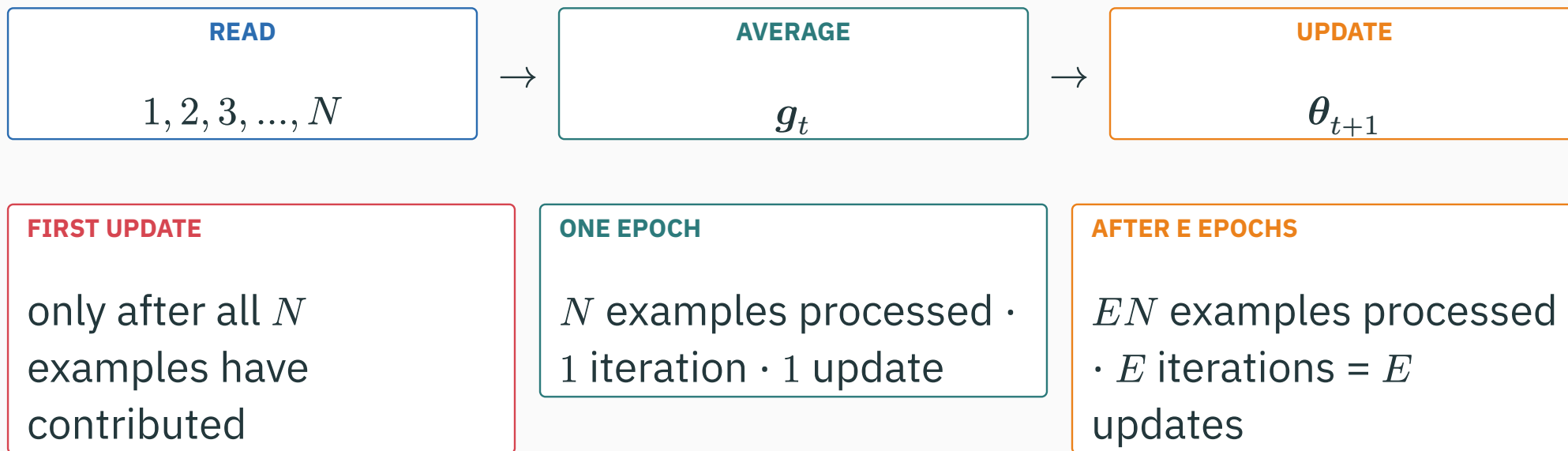
COMPUTED · THE FOUR DISPLAYED EXAMPLES · NOTEBOOK 00

| The contour point and the fitted line are two views of the same parameters (b, w) .

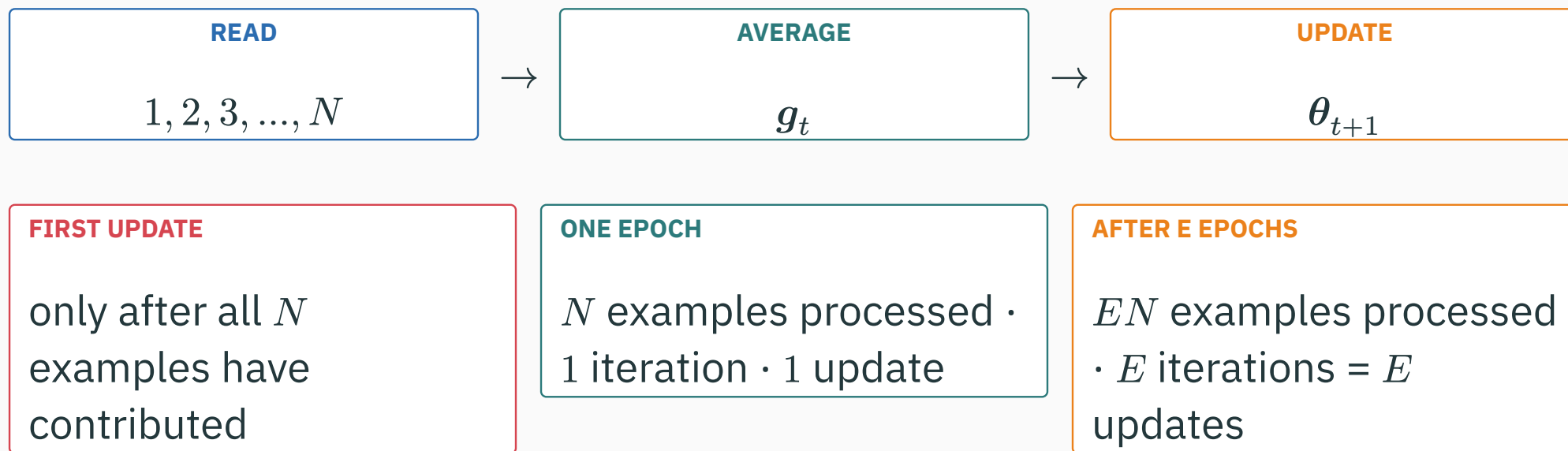
Batch gradient descent is exact—but every update must wait



Batch gradient descent is exact—but every update must wait



Batch gradient descent is exact—but every update must wait



Work versus time: one full-batch update processes N examples. Parallel hardware changes wall-clock time, but the parameters still wait for the reduction over all N contributions.

The training clock: examples, iterations, updates, epochs

N examples, $1 \leq B \leq N$ per batch $\rightarrow K = \lceil \frac{N}{B} \rceil$ iterations per epoch

$$\underbrace{\{3, 1\}}_{\text{iteration 1}} \quad \underbrace{\{4, 2\}}_{\text{iteration 2}}$$

Example: $N = 4, B = 2$. Both minibatches together process every example once—one epoch.

The training clock: examples, iterations, updates, epochs

D DERIVATION

N examples, $1 \leq B \leq N$ per batch $\rightarrow K = \lceil \frac{N}{B} \rceil$ iterations per epoch

$\underbrace{\{3, 1\}}_{\text{iteration 1}} \quad \underbrace{\{4, 2\}}_{\text{iteration 2}}$

Example: $N = 4, B = 2$. Both minibatches together process every example once—one epoch.

EPOCH one pass through N

ITERATION one minibatch loop body

UPDATE one parameter change

The training clock: examples, iterations, updates, epochs

N examples, $1 \leq B \leq N$ per batch $\rightarrow K = \lceil \frac{N}{B} \rceil$ iterations per epoch

$\underbrace{\{3, 1\}}_{\text{iteration 1}} \quad \underbrace{\{4, 2\}}_{\text{iteration 2}}$

Example: $N = 4, B = 2$. Both minibatches together process every example once—one epoch.

EPOCH one pass through N

ITERATION one minibatch loop body

UPDATE one parameter change

method	B	examples / iteration	iterations / epoch	updates / epoch
full batch	N	N	1	1
SGD	1	1	N	N
minibatch	B	B	$\lceil \frac{N}{B} \rceil$	$\lceil \frac{N}{B} \rceil$

Normally, iteration = update. After E epochs: EN examples and $E \lceil \frac{N}{B} \rceil$ updates. Gradient accumulation or skipped steps are exceptions.

FULL-BATCH GD

$$\mathbf{g}_t = \frac{1}{N} \sum_{i=1}^N \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t)$$

Exact for the current model—but all N contributions must arrive before one update.

SGD

$$I_t \sim \text{Uniform}\{1, \dots, N\}$$

$$\hat{\mathbf{g}}_t = \nabla_{\boldsymbol{\theta}} \ell_{I_t}(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \hat{\mathbf{g}}_t$$

FULL-BATCH GD

$$\mathbf{g}_t = \frac{1}{N} \sum_{i=1}^N \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t)$$

Exact for the current model—but all N contributions must arrive before one update.

SGD

$$I_t \sim \text{Uniform}\{1, \dots, N\}$$

$$\hat{\mathbf{g}}_t = \nabla_{\boldsymbol{\theta}} \ell_{I_t}(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \hat{\mathbf{g}}_t$$

Stochastic means the selected index I_t is random. The model and loss stay fixed; only the example used to estimate their gradient changes.

FULL-BATCH GD

$$\mathbf{g}_t = \frac{1}{N} \sum_{i=1}^N \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t)$$

Exact for the current model—but all N contributions must arrive before one update.

SGD

$$I_t \sim \text{Uniform}\{1, \dots, N\}$$

$$\hat{\mathbf{g}}_t = \nabla_{\boldsymbol{\theta}} \ell_{I_t}(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \hat{\mathbf{g}}_t$$

Stochastic means the selected index I_t is random. The model and loss stay fixed; only the example used to estimate their gradient changes.

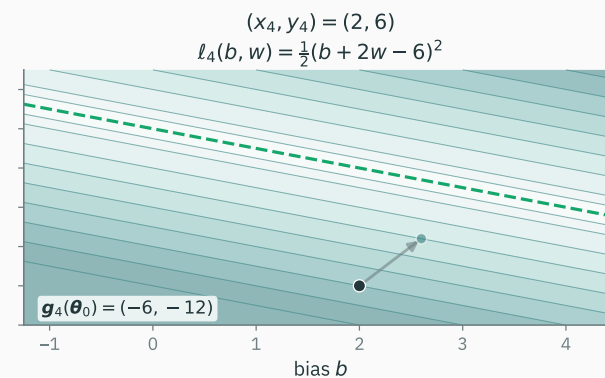
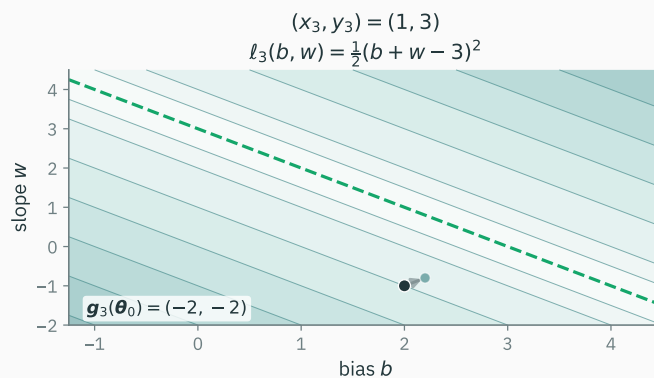
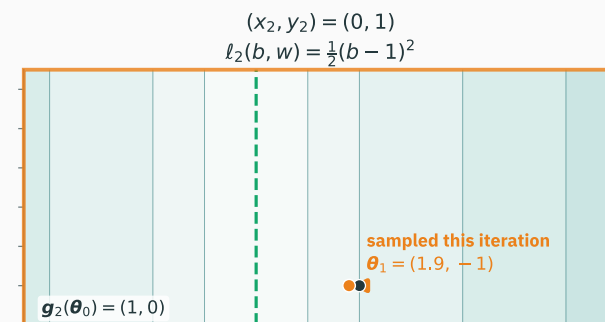
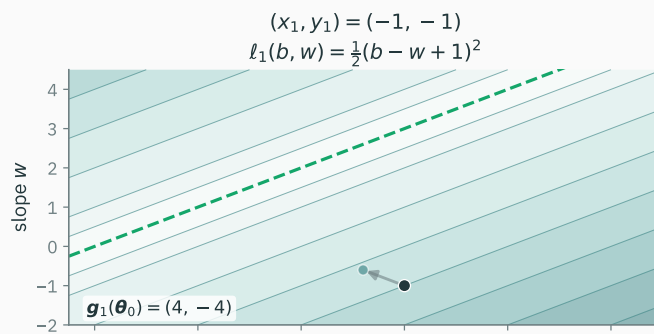
If $I_0 = 2$, then $(x_2, y_2) = (0, 1)$ and

$$\ell_2(b, w) = \frac{1}{2}(b - 1)^2, \quad \hat{\mathbf{g}}_0 = (2 - 1)(1, 0) = (1, 0), \quad \boldsymbol{\theta}_1 = (1.9, -1).$$

This step follows ℓ_2 . We have not yet shown what it does to the average loss $\mathcal{L}$.

Each example proposes a different update

$$\ell_i(b, w) = \frac{1}{2}(b + wx_i - y_i)^2, \quad \mathcal{L}(b, w) = \frac{1}{4} \sum_{i=1}^4 \ell_i(b, w).$$



COMPUTED · ORANGE = SAMPLED $I_0 = 2$ · MUTED ARROWS = ALTERNATIVES · FULL BATCH AVERAGES ALL FOUR

Worked check: ℓ_2 decreases, but $\mathcal{L}$ increases

D DERIVATION

1 One draw chooses the step

draw $I_0 = 2: (x_2, y_2) = (0, 1)$
 $\hat{y}_2 = 2 + (-1)(0) = 2$

$\hat{g}_0 = (2 - 1)(1, 0) = (1, 0)$

$\theta_1 = (2, -1) - 0.1(1, 0)$
 $= (1.9, -1)$

sampled $\ell_2: 0.500 \rightarrow 0.405 \downarrow$

$i = 1 \quad x = -1$

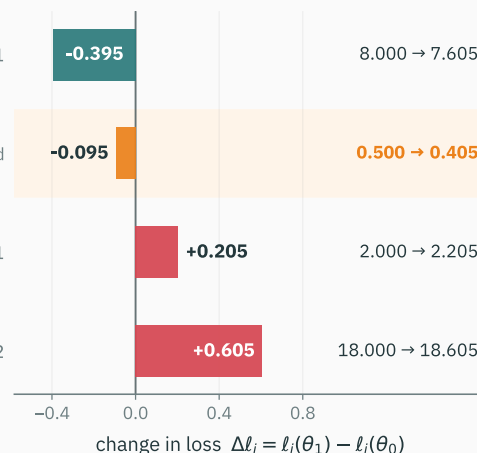
$i = 2 \quad x = 0$ sampled

$i = 3 \quad x = 1$

$i = 4 \quad x = 2$

2 Re-score all four examples

before → after



3 Average the changes

sampled: $\Delta \ell_2 = -0.095$

other three: $+0.415$

$$\sum_i \Delta \ell_i = +0.320$$

$$\Delta \mathcal{L} = +0.320/4 = +0.080$$

full $\mathcal{L}: 7.125 \rightarrow 7.205 \uparrow$

COMPUTED · ORANGE POINT SAMPLED · SAME FOUR-EXAMPLE FULL LOSS

The step is correct for ℓ_2 : $\Delta \ell_2 = -0.095$. Across all four examples, $\sum_i \Delta \ell_i = +0.320$, so $\Delta \mathcal{L} = +\frac{0.320}{4} = +0.080$.

A minibatch is the useful middle ground

For minibatch $\mathcal{B}_t$, let $B_t = |\mathcal{B}_t|$ (the final batch may be smaller):

$$\hat{\mathbf{g}}_t = \frac{1}{B_t} \sum_{i \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t).$$

A minibatch is the useful middle ground

For minibatch $\mathcal{B}_t$, let $B_t = |\mathcal{B}_t|$ (the final batch may be smaller):

$$\hat{\mathbf{g}}_t = \frac{1}{B_t} \sum_{i \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t).$$

If $\mathcal{B}_0 = \{1, 2\}$ contains the first two gradients in our table,

$$\hat{\mathbf{g}}_0 = \frac{1}{2}[(4, -4) + (1, 0)] = (2.5, -2).$$

A minibatch is the useful middle ground

For minibatch $\mathcal{B}_t$, let $B_t = |\mathcal{B}_t|$ (the final batch may be smaller):

$$\hat{\mathbf{g}}_t = \frac{1}{B_t} \sum_{i \in \mathcal{B}_t} \nabla_{\boldsymbol{\theta}} \ell_i(\boldsymbol{\theta}_t).$$

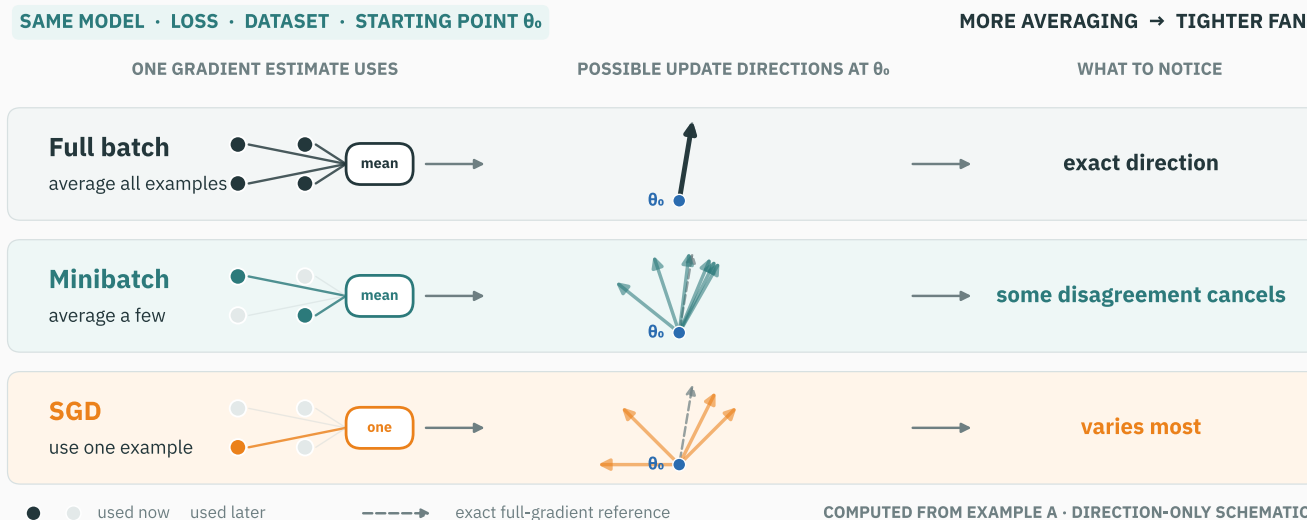
If $\mathcal{B}_0 = \{1, 2\}$ contains the first two gradients in our table,

$$\hat{\mathbf{g}}_0 = \frac{1}{2}[(4, -4) + (1, 0)] = (2.5, -2).$$

$$\boldsymbol{\theta}_1 = (2, -1) - 0.1(2.5, -2) = (1.75, -0.8).$$

Batch size is one knob: $B_t = 1$ is **SGD**; $B_t = N$ is **full-batch GD**. Next: **what does averaging buy us?**

Why minibatches help: averaging cancels some disagreement



COMPUTED FROM EXAMPLE A · FIXED MODEL, DATA, AND PARAMETERS · ONLY THE AVERAGING CHANGES

One example can pull in an idiosyncratic direction. Averaging a few cancels some disagreement; averaging all examples leaves the exact full-data direction.

Batch, stochastic, and minibatch GD—qualitatively

	Full batch	SGD	Minibatch
examples / update	N	1	B
updates / epoch	1	N	$\lceil \frac{N}{B} \rceil$
gradient noise	none	highest	intermediate
parallelism	high but large	poor	usually efficient
near optimum; fixed $\eta > 0$	smooth	typically wanders if sampling noise remains	less wandering as B grows

In deep learning, “SGD” often informally means the whole minibatch-SGD family. The batch size states which member is actually used.

Choosing the batch size in PyTorch

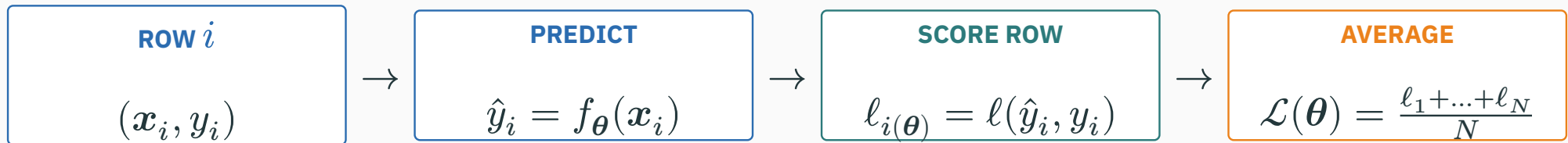
```
loader = DataLoader(dataset, batch_size=B, shuffle=True)

for epoch in range(E):
    for xb, yb in loader:          # one iteration / minibatch
        optimizer.zero_grad(set_to_none=True)
        pred = model(xb)
        loss = 0.5 * (pred - yb).square().mean()
        loss.backward()           # gradient of this batch mean
        optimizer.step()          # one parameter update
```

`torch.optim.SGD` transforms the gradient supplied by the loop; its name does **not** imply $B = 1$. The factor $\frac{1}{2}$ matches our analytic half-MSE convention.

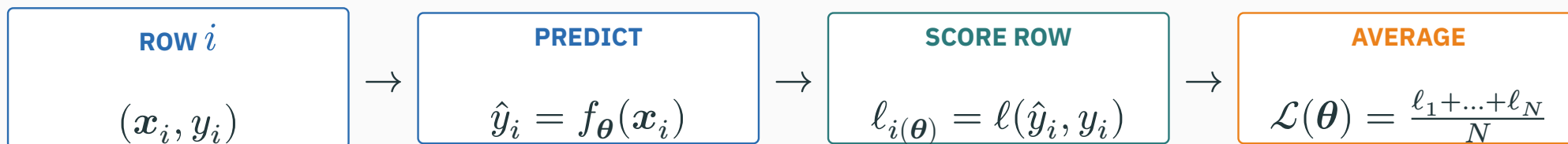
**How can one example estimate
the full gradient?**

From data rows to the full loss



For Example A,

$$\ell_{i(b,w)} = \frac{1}{2}(b + wx_i - y_i)^2, \quad \mathcal{L}(b, w) = \frac{1}{4} \sum_{i=1}^4 \ell_{i(b,w)}.$$



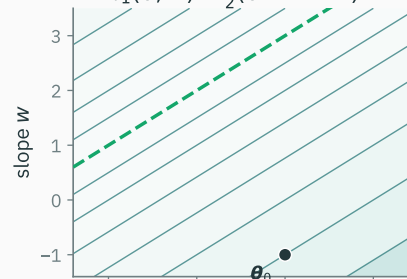
For Example A,

$$\ell_{i(b,w)} = \frac{1}{2}(b + wx_i - y_i)^2, \quad \mathcal{L}(b, w) = \frac{1}{4} \sum_{i=1}^4 \ell_{i(b,w)}.$$

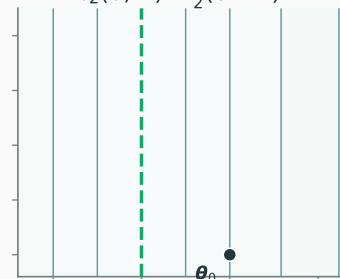
The full loss is an average of per-example losses. Therefore its gradient is the same average: $\nabla \mathcal{L} = \frac{1}{N} \sum_i \nabla \ell_i$.

Each example defines its own loss surface

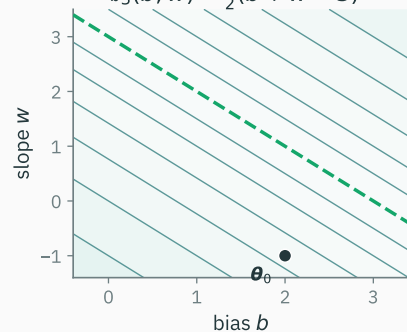
Example 1: $(x_1, y_1) = (-1, -1)$
 $\ell_1(b, w) = \frac{1}{2}(b - w + 1)^2$



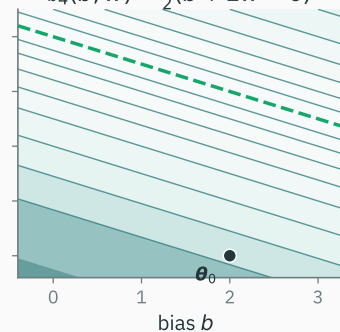
Example 2: $(x_2, y_2) = (0, 1)$
 $\ell_2(b, w) = \frac{1}{2}(b - 1)^2$



Example 3: $(x_3, y_3) = (1, 3)$
 $\ell_3(b, w) = \frac{1}{2}(b + w - 3)^2$



Example 4: $(x_4, y_4) = (2, 6)$
 $\ell_4(b, w) = \frac{1}{2}(b + 2w - 6)^2$

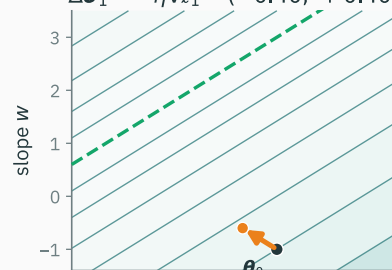


same model $\theta_0 = (2, -1)$ in every panel · dashed green: zero-loss line for that one example

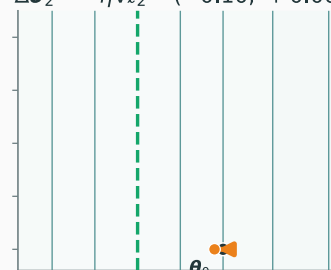
COMPUTED FROM THE FOUR DISPLAYED EXAMPLE A ROWS · IDENTICAL AXES AND CONTOUR LEVELS

Same parameters, different descent directions

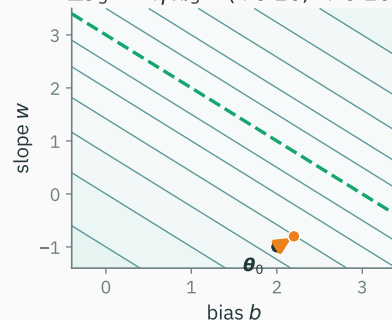
Example 1: $\ell_1(b, w) = \frac{1}{2}(b - w + 1)^2$
 $\Delta\theta_1 = -\eta\nabla\ell_1 = (-0.40, +0.40)$



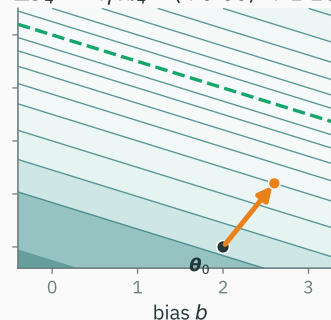
Example 2: $\ell_2(b, w) = \frac{1}{2}(b - 1)^2$
 $\Delta\theta_2 = -\eta\nabla\ell_2 = (-0.10, +0.00)$



Example 3: $\ell_3(b, w) = \frac{1}{2}(b + w - 3)^2$
 $\Delta\theta_3 = -\eta\nabla\ell_3 = (+0.20, +0.20)$



Example 4: $\ell_4(b, w) = \frac{1}{2}(b + 2w - 6)^2$
 $\Delta\theta_4 = -\eta\nabla\ell_4 = (+0.60, +1.20)$

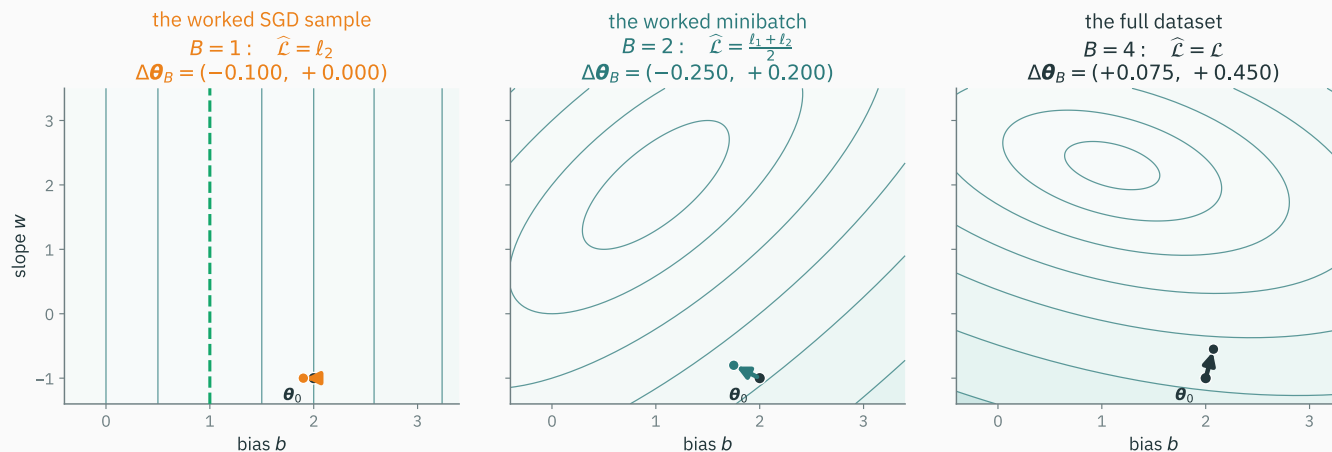


orange: the exact parameter update for that panel, with $\eta = 0.10$ · coordinates are $(\Delta b, \Delta w)$

COMPUTED FROM EXAMPLE A · SAME $\theta_0 = (2, -1)$ AND $\eta = 0.1$ IN EVERY PANEL

Each arrow is the **update** $\Delta\theta_i = -\eta\nabla\ell_i(\theta_0)$, so it points downhill on that panel's loss.

A minibatch averages the selected loss surfaces



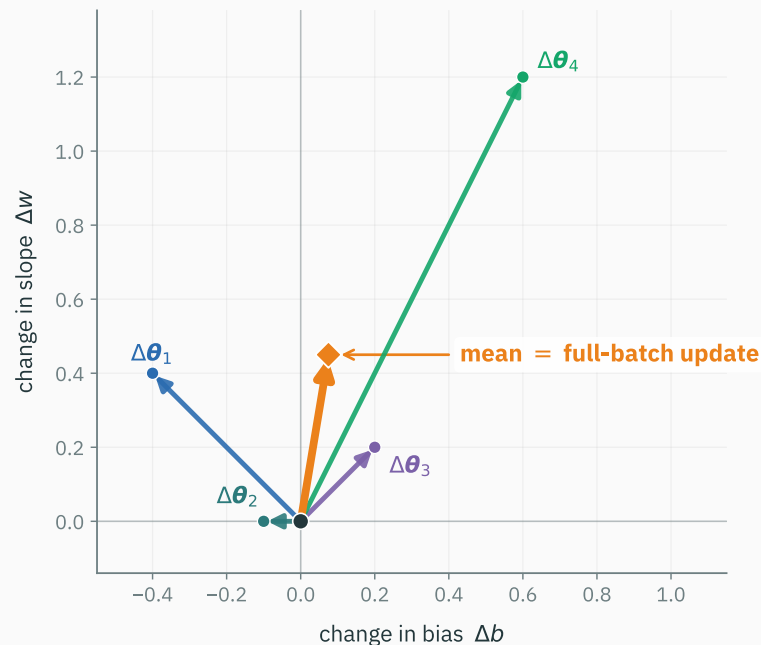
same θ_0 and $\eta = 0.10$ · averaging examples changes both the loss surface and its gradient

COMPUTED FROM EXAMPLE A · ROW 2, ROWS 1-2, AND ALL FOUR ROWS

$$\mathcal{L}_{\mathcal{B}}(\theta) = \frac{1}{B} \sum_{i \in \mathcal{B}} \ell_i(\theta) \quad \rightarrow \quad \nabla \mathcal{L}_{\mathcal{B}} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell_i.$$

Increasing B averages more troughs. Their disagreements partly cancel, and the full batch leaves one exact empirical-loss surface.

Average the one-example updates



COMPUTED FROM EXAMPLE A AT FIXED θ_0

Translate the four updates to a common origin, then average coordinates:

$$\frac{1}{4} \sum_{i=1}^4 \Delta\theta_i = (0.075, 0.450).$$

$$-\frac{\eta}{4} \sum_{i=1}^4 \nabla \ell_i(\theta_0) = \Delta\theta_{\text{full}}.$$

Different samples give different updates; their arithmetic mean is the full-batch update.

An estimator maps random data to an estimate

For a generic discrete random variable X with probability mass $p(x)$, expectation is the probability-weighted average

$$\mathbb{E}[f(X)] = \sum_{x \in \mathcal{X}} p(x) f(x).$$

RANDOM DRAW

$X = x$ occurs with probability $p(x)$

ESTIMATE

apply the rule f and observe $f(X)$

TARGET

a fixed quantity μ to be estimated

The estimator $f(X)$ is **unbiased** for μ when $\mathbb{E}[f(X)] = \mu$.

For continuous X , replace the sum by an integral. Unbiasedness describes the centre across repeated draws—not one draw.

Hold θ_t fixed and substitute into the generic expectation:

$$X \rightarrow I_t, \quad p(i) = \mathbb{P}(I_t = i \mid \theta_t) = \frac{1}{N}, \quad f(i) = \nabla_{\theta} \ell_{i(\theta_t)}.$$

The stochastic gradient estimator is the gradient of the sampled loss:

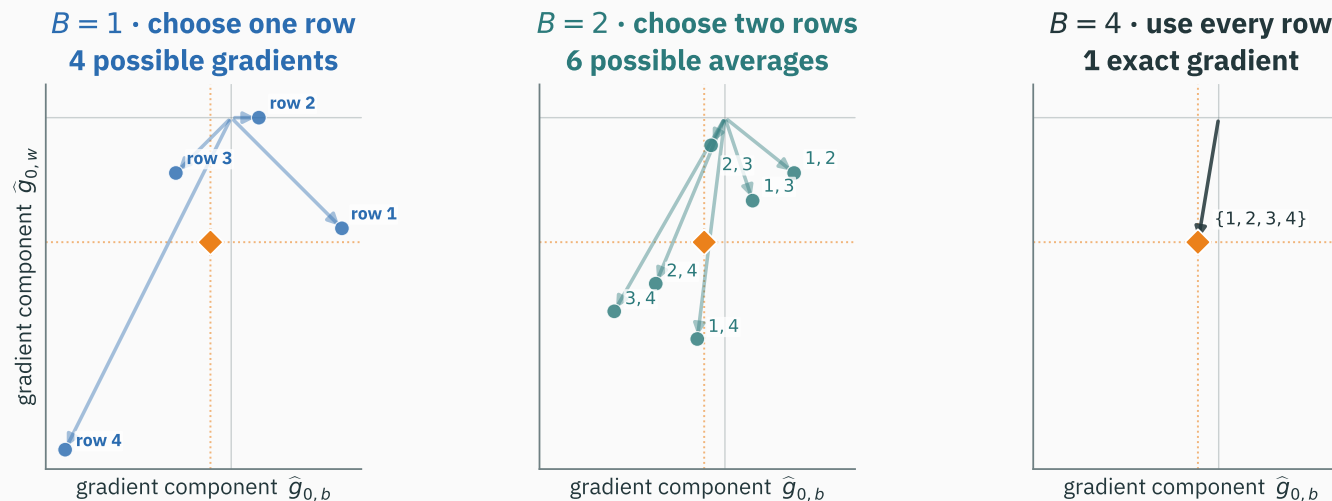
$$\hat{g}_t = f(I_t) = \nabla_{\theta} \ell_{I_t}(\theta_t).$$

$$\mathbb{E}[\hat{g}_t \mid \theta_t] = \sum_{i=1}^N p(i) f(i) = \sum_{i=1}^N \frac{1}{N} \nabla_{\theta} \ell_{i(\theta_t)} = \nabla \left[\frac{1}{N} \sum_{i=1}^N \ell_{i(\theta_t)} \right] = g_t.$$

For Example A, the mean of $(4, -4), (1, 0), (-2, -2), (-6, -12)$ is $(-0.75, -4.5)$ —the full gradient computed earlier.

Minibatches preserve the mean and reduce the spread

A minibatch estimator is simply an **average of one-row estimators**: $\hat{g}_{B,t} = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla \ell_{i(\theta_t)}$.



labels identify the selected row(s) · orange diamond: the same mean = g_0 in every panel

EXACT ENUMERATION OF EVERY UNIFORM EXAMPLE A SUBSET AT THE SAME θ_0

Same centre: $\mathbb{E}[\hat{g}_{B,t} \mid \theta_t] = g_t$. **Less spread:** larger B cancels more disagreement.

Unbiased means correct on average—not downhill every time

Our earlier valid SGD draw sampled row 2 and used $\hat{g}_0 = (1, 0)$, so $\theta_1 = (1.9, -1)$.

SAMPLED LOSS

$$\ell_2 : 0.500 \rightarrow 0.405$$

The update succeeds on the sampled row.

FULL LOSS

$$\mathcal{L} : 7.125 \rightarrow 7.205$$

The same update is uphill for the average loss.

Unbiasedness guarantees $\mathbb{E}[\hat{g}_t \mid \theta_t] = g_t$ —equivalently, $\mathbb{E}[\Delta\theta_t \mid \theta_t] = -\eta g_t$. It does not make every estimate close, every step downhill, or convergence automatic.

1 • INSPECT

Plot each ℓ_i , a minibatch average, and the full $\mathcal{L}$.

2 • VERIFY

Enumerate the possible gradients and check their mean.

3 • EXPERIMENT

Change B and watch the estimator cloud contract.

INTERACTIVE

↗ colab.research.google.com/github/nipunbatra/dl-teaching/blob/master/notebooks/L05/00_full_batch_sgd_minibatch.ipynb

Open **Full batch, SGD, minibatches, and gradient estimators** in Colab ↗

**An exact gradient can still be
slow**

Example B: one feature is ten times larger

Use $\hat{y}_i = \theta_1 x_{i1} + \theta_2 x_{i2}$ and generate targets with $\theta^* = (1, -0.5)$.

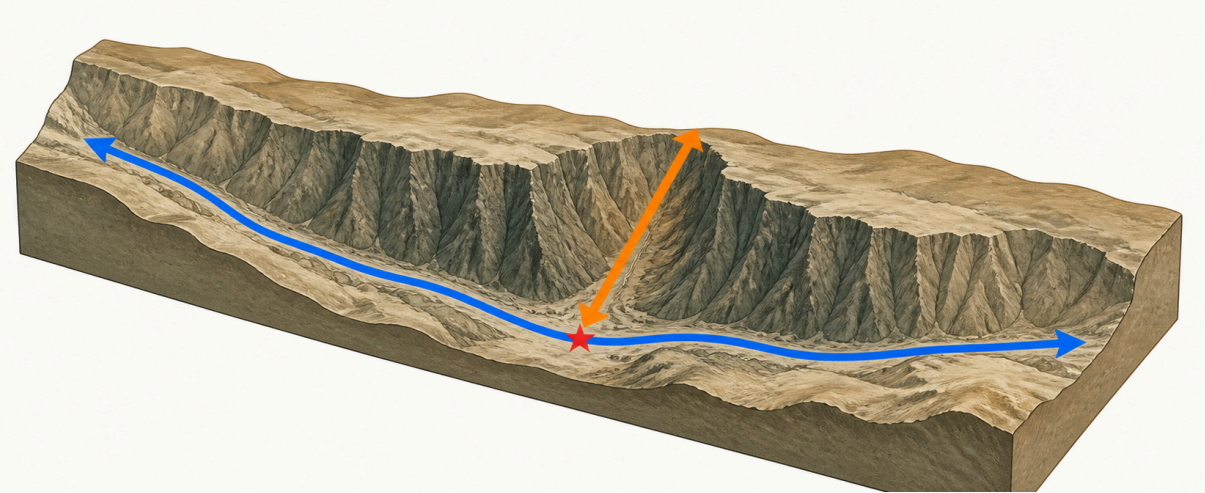
The first feature is ± 1 ; the second is ± 10 .
This deliberate scale difference creates a transparent optimization problem.

i	x_{i1}	x_{i2}	y_i
1	-1	-10	4
2	-1	10	-6
3	1	-10	6
4	1	10	-4

Gradient descent starts at $\theta_0 = (-1.4, 0.35)$ and moves toward the optimum $\theta^* = (1, -0.5)$.

A ravine is a long, narrow valley with steep walls

A **valley** is low ground between higher sides. A **ravine** is a long, narrow valley whose side walls are steep.



Along the valley follows the low floor; height changes gently.

Across the valley crosses the walls; height changes quickly.

ILLUSTRATIVE TERRAIN ANALOGY · GENERATED FOR THIS LECTURE

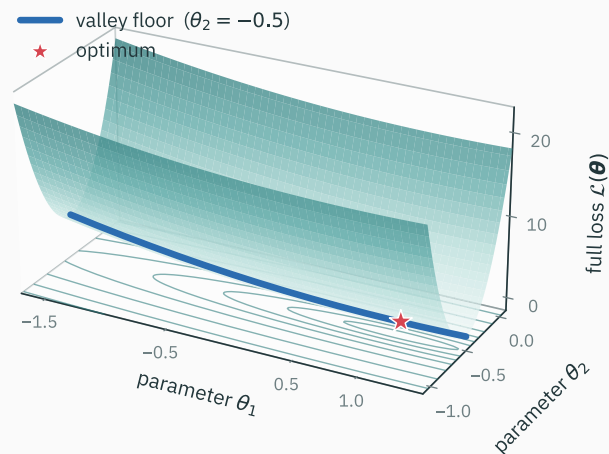
The $10 \times$ feature turns the loss into a ravine

Averaging the four rows gives an exact full-batch loss:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}(\theta_1 - 1)^2 + 50(\theta_2 + 0.5)^2, \quad \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = (\theta_1 - 1, 100(\theta_2 + 0.5)).$$

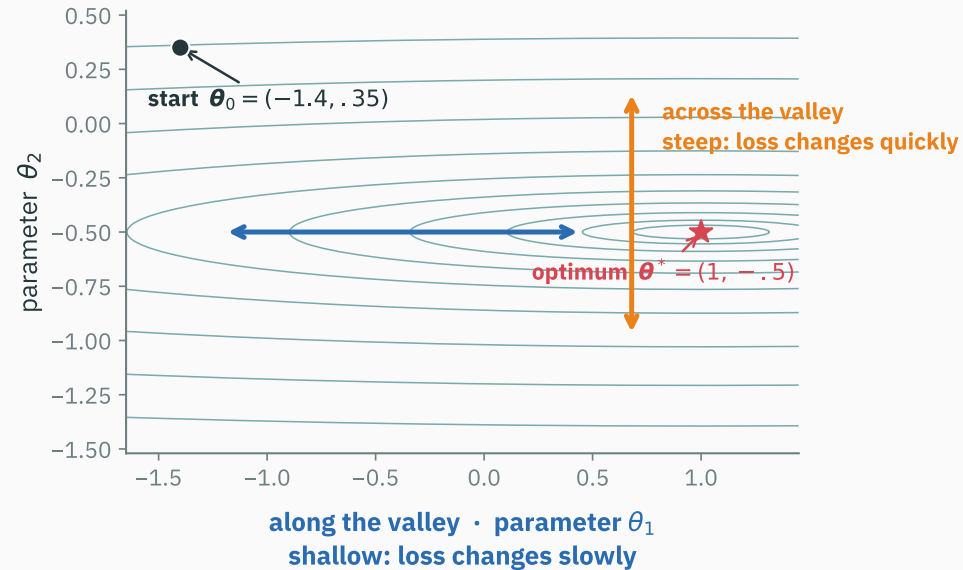
ALONG the valley $\cdot \theta_1 \cdot$ loss changes gently

ACROSS the valley $\cdot \theta_2 \cdot$ loss changes steeply



COMPUTED FROM THE DISPLAYED FULL-BATCH LOSS $\cdot$ NATIVE PARAMETER COORDINATES

Contours show the same surface from above



computed from the four displayed rows · each contour joins points with the same full-batch loss

COMPUTED FROM THE FOUR DISPLAYED REGRESSION EXAMPLES · EXACT FULL-BATCH LOSS

Each contour joins parameter settings with the same loss. Widely spaced contours mean gentle change; tightly packed contours mean steep change.

One correct step can overshoot across the valley

At $\theta_0 = (-1.4, .35)$,

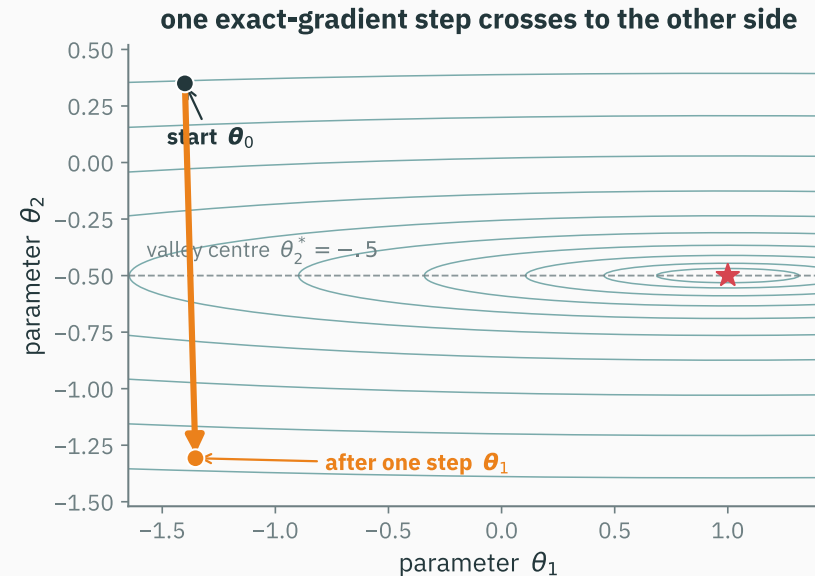
$$\begin{aligned} g_0 &= \nabla_{\theta} \mathcal{L}(\theta_0) \\ &= (-1.4 - 1, 100(.35 + .5)) = (-2.4, 85). \end{aligned}$$

With $\eta = .0195$,

$$\begin{aligned} \Delta\theta_0 &= -\eta g_0 \\ &= -.0195(-2.4, 85) = (.0468, -1.6575). \end{aligned}$$

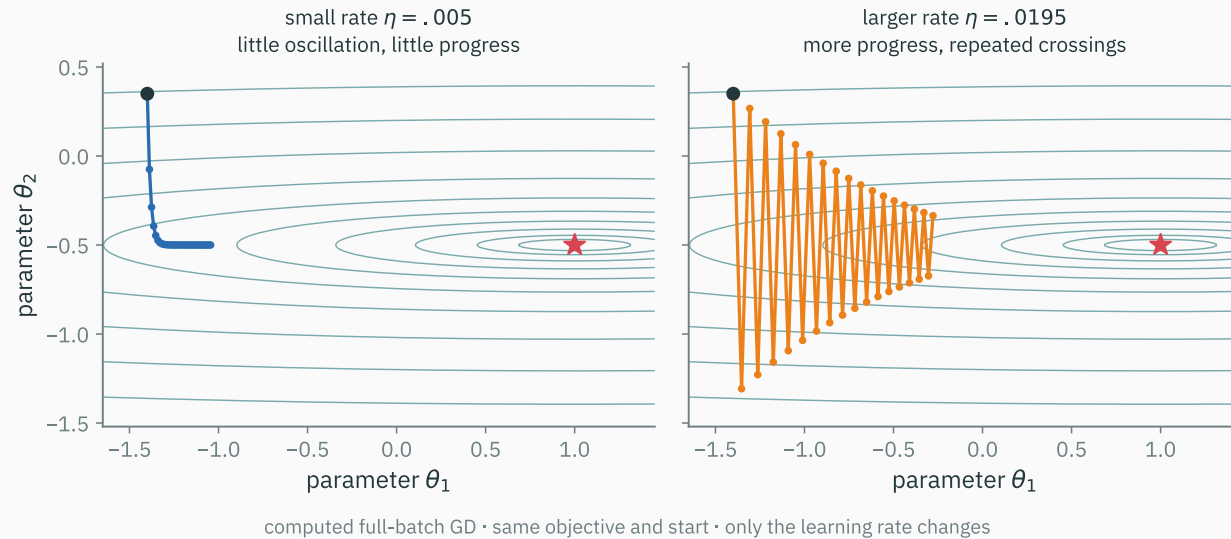
$$\theta_1 = \theta_0 + \Delta\theta_0 = (-1.3532, -1.3075).$$

g_0 points uphill; $-g_0$ moves downhill and crosses $\theta_2^* = -.5$.



COMPUTED EXACT-GRADIENT UPDATE

The trade-off is visible in the computed paths



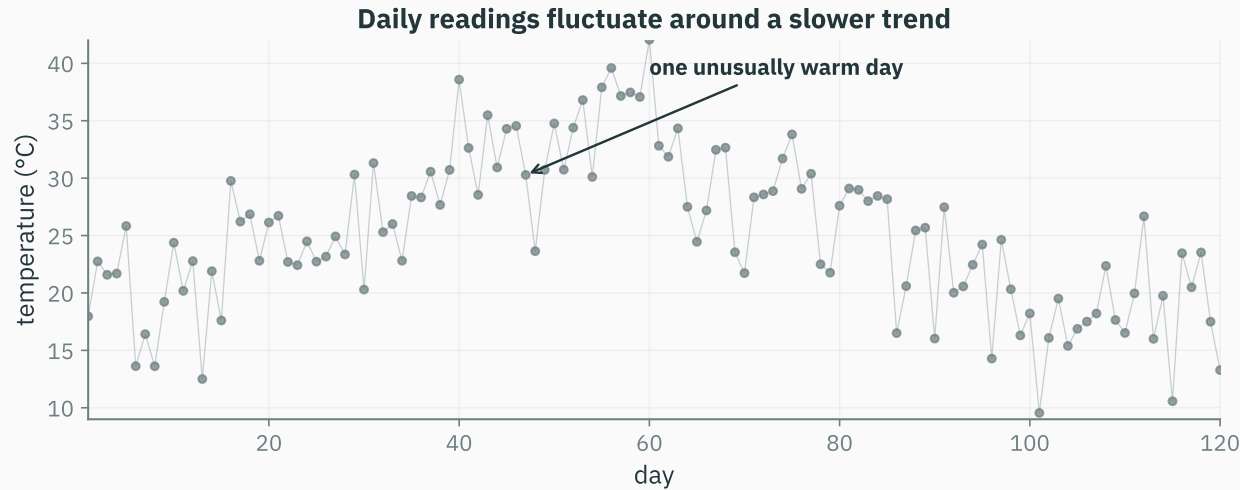
COMPUTED FULL-BATCH GD · SAME DATA, START, AND UPDATE BUDGET

The small rate avoids crossing $\theta_2^* = -0.5$, but barely changes θ_1 . The larger rate moves θ_1 faster and repeatedly crosses the optimum in θ_2 . One global learning rate creates this trade-off.

**How can we smooth a noisy
sequence?**

Daily temperature readings are noisy

I INTERACTIVE

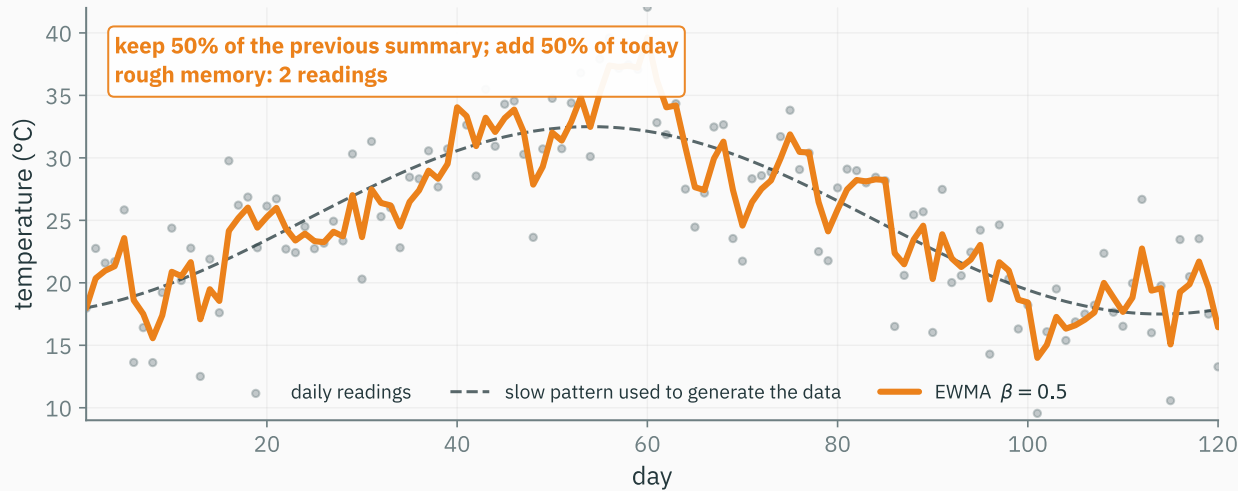


CONSTRUCTED 120-DAY TEMPERATURE SIGNAL • FIXED SEED 12

A single day may be unusually warm or cool. To follow the slower trend, we keep one running average and update it as each new reading arrives.

Open the [EWMA + Momentum Explorer](#) ↗ — change β and watch the smoothed curve.

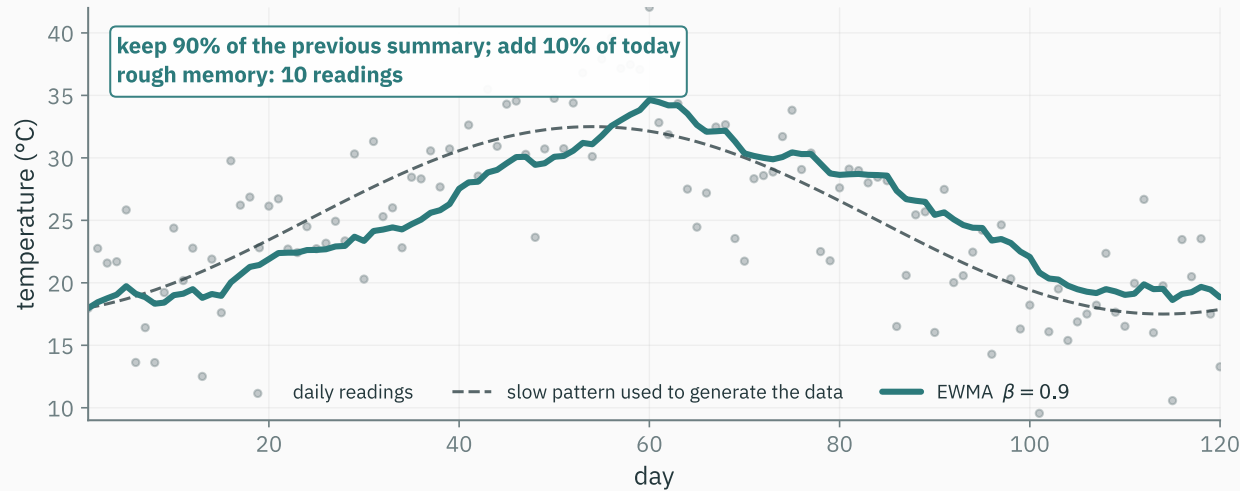
Short memory reacts quickly—and follows the noise



SAME READINGS • EWMA INITIALIZED AT THE FIRST READING

β is the fraction of the previous summary retained. At $\beta = .5$, the previous summary and today's reading receive equal weight, so the curve remains jumpy.

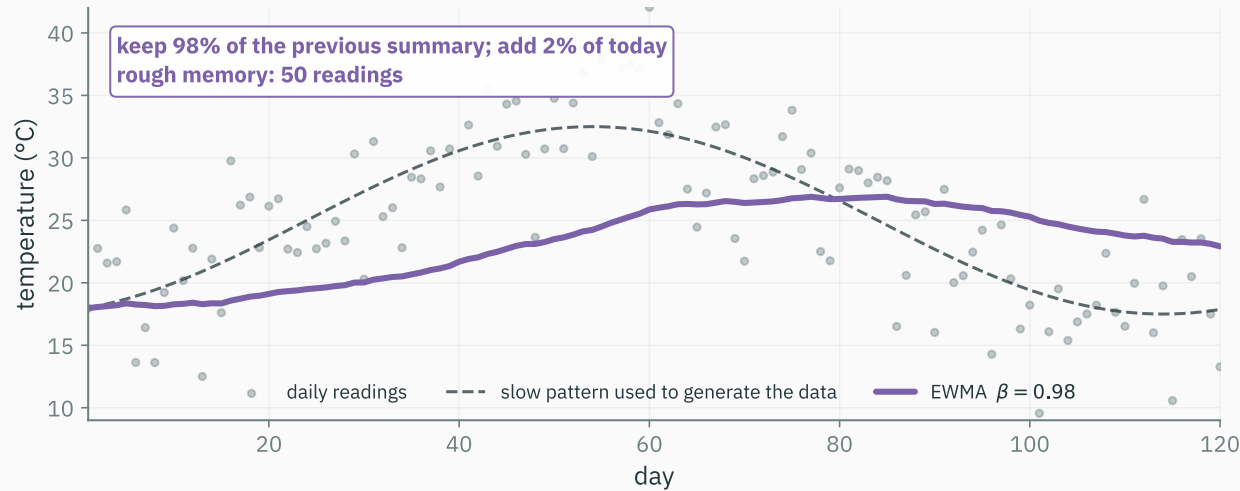
More memory gives a useful compromise



SAME READINGS · ONLY β CHANGES

At $\beta = .9$, keep 90% of the previous summary and add 10% of today's reading.
Noise is reduced, with some delay when the trend turns.

Too much memory becomes slow to respond



SAME READINGS · ONLY β CHANGES

$\beta = .98$ averages over a much longer effective history. It is very smooth, but it visibly lags the changing signal.

$$\underbrace{m_t}_{\text{running memory after today's reading}} = \underbrace{\beta m_{t-1}}_{\text{keep the previous summary}} + \underbrace{(1 - \beta)x_t}_{\text{add today's observation}}, \quad 0 \leq \beta < 1.$$

ONE WORKED UPDATE

$$m_{t-1} = 30^\circ C, x_t = 36^\circ C, \beta = .9$$

$$m_t = .9(30) + .1(36) = 30.6^\circ C$$

WHAT IS CARRIED FORWARD?

Keep the new memory $m_t = 30.6$ for the next observation. The earlier readings need not be stored.

This signal summary is an **exponentially weighted moving average (EWMA). Optimizer momentum uses the same normalized memory idea; its complete update rule is boxed before we substitute any values. PyTorch's differently scaled buffer is discussed only after the derivation.**

1. Write the current update.

$$m_t = \beta m_{t-1} + (1 - \beta)x_t.$$

2. Write the same update one step earlier.

$$m_{t-1} = \beta m_{t-2} + (1 - \beta)x_{t-1}.$$

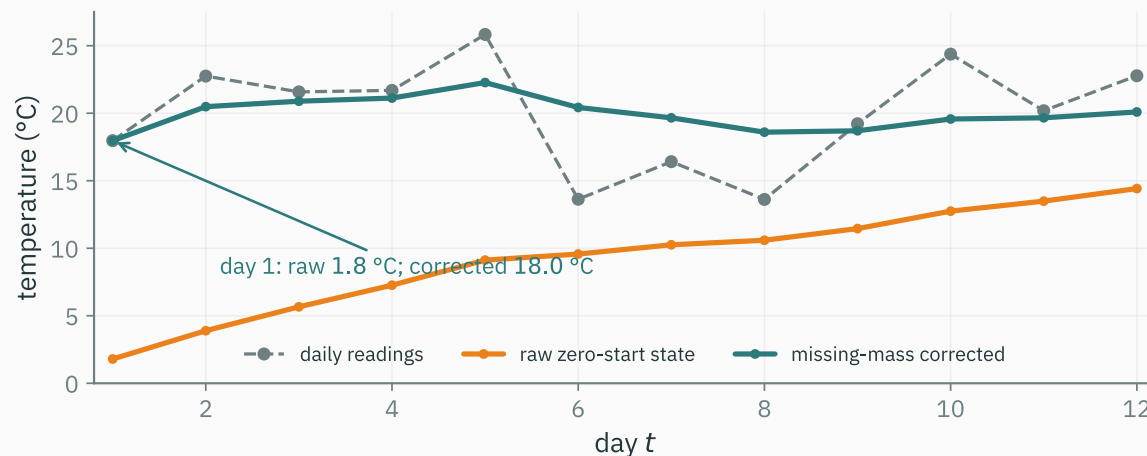
3. Substitute this expression for m_{t-1} .

$$m_t = \beta[\beta m_{t-2} + (1 - \beta)x_{t-1}] + (1 - \beta)x_t$$

$$m_t = (1 - \beta)x_t + (1 - \beta)\beta x_{t-1} + \beta^2 m_{t-2}.$$

Each step backwards adds one factor of β . For the zero-start recurrence, x_{t-k} has weight $(1 - \beta)\beta^k$. For $\beta = .9$, these weights begin .100, .090, .081, .0729, In the plotted warm start, x_1 instead carries the remaining weight β^{t-1} .

Starting from zero makes the first temperature estimate too small



SAME FIXED-SEED TEMPERATURE READINGS $\cdot \beta = .9$

With the arbitrary initial state $m_0 = 0^\circ C$ and the same first reading $x_1 = 17.968^\circ C$, $m_1 = .9(0) + .1(17.968) = 1.7968^\circ C \approx 1.80^\circ C$. The remaining .9 weight came from the chosen initial zero—not from a temperature observation.

Divide by the weight that has arrived

$$w_t = (1 - \beta) \sum_{k=0}^{t-1} \beta^k = 1 - \beta^t$$

weight from observations 1 through t

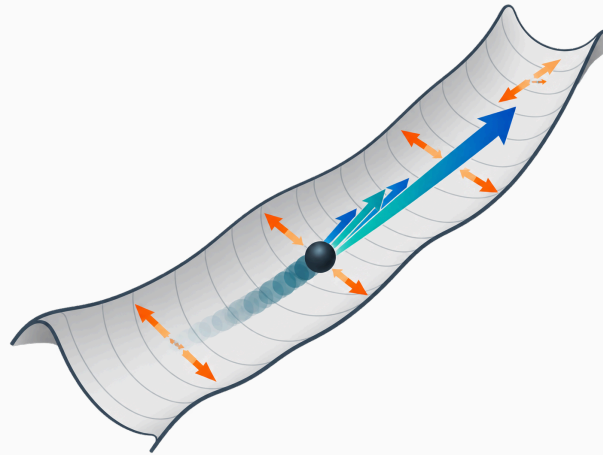
$$\hat{m}_t = \frac{m_t}{w_t} = \frac{m_t}{1 - \beta^t}$$

normalize the raw zero-start memory

$$\textbf{Day 1: } \hat{m}_1 = \frac{1.7968}{1-.9} = 17.968^{\circ}C \approx 17.97^{\circ}C.$$

The earlier temperature curves set $m_1 = x_1$, so they did not have this cold start. Dividing by the weight that has arrived removes the artificial pull toward zero; it does not remove measurement noise.

Is this direction consistent over time?



CONCEPTUAL ILLUSTRATION · GENERATED WITH OPENAI IMAGEN · NOT DATA

Momentum averages **signed** gradients. Components that keep the same sign survive in the memory; components that repeatedly reverse partly cancel. The result keeps persistent motion while damping the side-to-side zigzag.

GRADIENT DESCENT WITH MOMENTUM

$$\hat{\mathbf{g}}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathcal{B}_t}(\boldsymbol{\theta}_t)$$

$$\mathbf{m}_{t+1} = \beta \mathbf{m}_t + (1 - \beta) \hat{\mathbf{g}}_t, \quad \mathbf{m}_0 = \mathbf{0}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{m}_{t+1}$$

$\hat{\mathbf{g}}_t$ is the gradient measured on the current full batch or minibatch. $\mathbf{m}_t$ is an EWMA of present and past gradients: β keeps the old memory and $1 - \beta$ weights the new gradient, exactly as in the temperature example. η controls the move. When $\beta = 0$, momentum reduces to ordinary gradient descent.

First update: the same gradient, two update rules

At $\theta_0 = (-1.4, .35)$, both methods start from $\mathcal{L}(\theta_0) = 39.005$ and measure

$$g_0 = (\theta_{0,1} - 1, 100(\theta_{0,2} + .5)) = (-2.4, 85).$$

PLAIN GRADIENT DESCENT

$$\Delta\theta_0 = -\eta g_0 = -.0195(-2.4, 85) = (.0468, -1.6575)$$

$$\theta_1 = \theta_0 + \Delta\theta_0 = (-1.4, .35) + (.0468, -1.6575) = (-1.3532, -1.3075)$$

$$\mathcal{L}(\theta_1) = \frac{1}{2}(-2.3532)^2 + 50(-.8075)^2 \approx 35.372$$

GRADIENT DESCENT WITH MOMENTUM

$$m_1 = .25m_0 + .75g_0 = .25(0, 0) + .75(-2.4, 85) = (-1.8, 63.75)$$

$$\Delta\theta_0 = -\eta m_1 = (.0351, -1.243125)$$

$$\theta_1 = (-1.4, .35) + (.0351, -1.243125) = (-1.3649, -.893125)$$

$$\mathcal{L}(\theta_1) = \frac{1}{2}(-2.3649)^2 + 50(-.393125)^2 \approx 10.524$$

Updates 2 and 3: repeat the same three equations

Each method first measures a new gradient at **its own** current parameters.

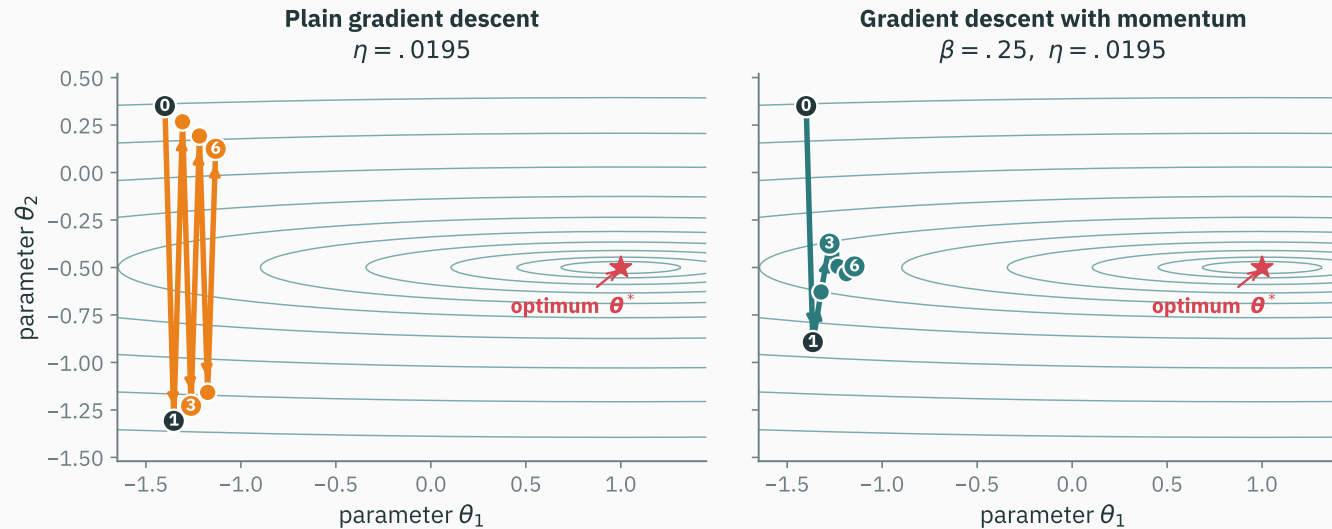
update	plain GD	momentum
2	$g_1 = (-2.3532, -80.75)$ $\Delta\theta = -.0195g_1 = (.0459, 1.5746)$ $\theta_2 = (-1.3073, .2671)$ $\mathcal{L}_2 = 32.086$	$g_1 = (-2.3649, -39.3125)$ $m_2 = .25m_1 + .75g_1 =$ $(-2.2237, -13.5469)$ $\Delta\theta = -.0195m_2 = (.0434, .2642)$ $\theta_2 = (-1.3215, -.6290), \mathcal{L}_2 = 3.526$
3	$g_2 = (-2.3073, 76.7125)$ $\Delta\theta = -.0195g_2 = (.0450, -1.4959)$ $\theta_3 = (-1.2623, -1.2288)$ $\mathcal{L}_3 = 29.114$	$g_2 = (-2.3215, -12.8961)$ $m_3 = .25m_2 + .75g_2 =$ $(-2.2971, -13.0588)$ $\Delta\theta = -.0195m_3 = (.0448, .2546)$ $\theta_3 = (-1.2767, -.3743), \mathcal{L}_3 = 3.382$

Updates 4–6: the same learning rate, different moves

Each method recomputes g_t at its own current θ_t .

update	plain GD	momentum
4	$\Delta\theta = (.0441, 1.4211)$ $\theta_4 = (-1.2182, .1923)$ $\mathcal{L}_4 = 26.426$	$m_4 = (-2.2818, 6.1617)$ $\Delta\theta = (.0445, -.1202)$ $\theta_4 = (-1.2322, -.4945), \mathcal{L}_4 = 2.493$
5	$\Delta\theta = (.0433, -1.3500)$ $\theta_5 = (-1.1749, -1.1577)$ $\mathcal{L}_5 = 23.995$	$m_5 = (-2.2446, 1.9553)$ $\Delta\theta = (.0438, -.0381)$ $\theta_5 = (-1.1885, -.5326), \mathcal{L}_5 = 2.448$
6	$\Delta\theta = (.0424, 1.2825)$ $\theta_6 = (-1.1325, .1248)$ $\mathcal{L}_6 = 21.794$	$m_6 = (-2.2025, -1.9559)$ $\Delta\theta = (.0429, .0381)$ $\theta_6 = (-1.1455, -.4945), \mathcal{L}_6 = 2.303$

The same learning rate produces very different paths



After 6 exact gradients, momentum is near the valley centre: $\theta_2 = -.494$ versus $.125$.
computed from Example B · same start and $\eta = .0195$ · normalized EWMA momentum · updates 0–6

COMPUTED ON EXAMPLE B · POINTS 0–6 · EXACT FULL-BATCH GRADIENTS

Both use $\eta = .0195$. Plain GD follows each current sign reversal; momentum averages it with recent gradients before moving.

Momentum update 2 in θ_1 : both pushes point right

D DERIVATION

At momentum's θ_1 , the first coordinate has memory $m_{1,1} = -1.8$ and new gradient $g_{1,1} = -2.3649$. Expand the two contributions inside $m_2 = .25m_1 + .75g_1$:

STORED HISTORY

$$-.0195[.25(-1.8)] = +.0088$$

$$+.0088 \rightarrow$$

NEW GRADIENT

$$-.0195[.75(-2.3649)] = +.0346$$

$$+.0346 \rightarrow$$

$$+.0088 + .0346 = +.0434 \rightarrow$$

$$\text{first coordinate: } -1.3649 + .0434 = -1.3215$$

The old and new gradients have the same sign, so their contributions add. The memory keeps a persistent rightward direction.

Update 2 in θ_2 : the old and new pushes nearly cancel

For the second coordinate, the stored memory is $m_{1,2} = 63.75$, while the new gradient has reversed sign: $g_{1,2} = -39.3125$.

STORED HISTORY

$$-.0195[.25(63.75)] = -.3108$$

$$-.3108 \downarrow$$

NEW GRADIENT

$$-.0195[.75(-39.3125)] = +.5750$$

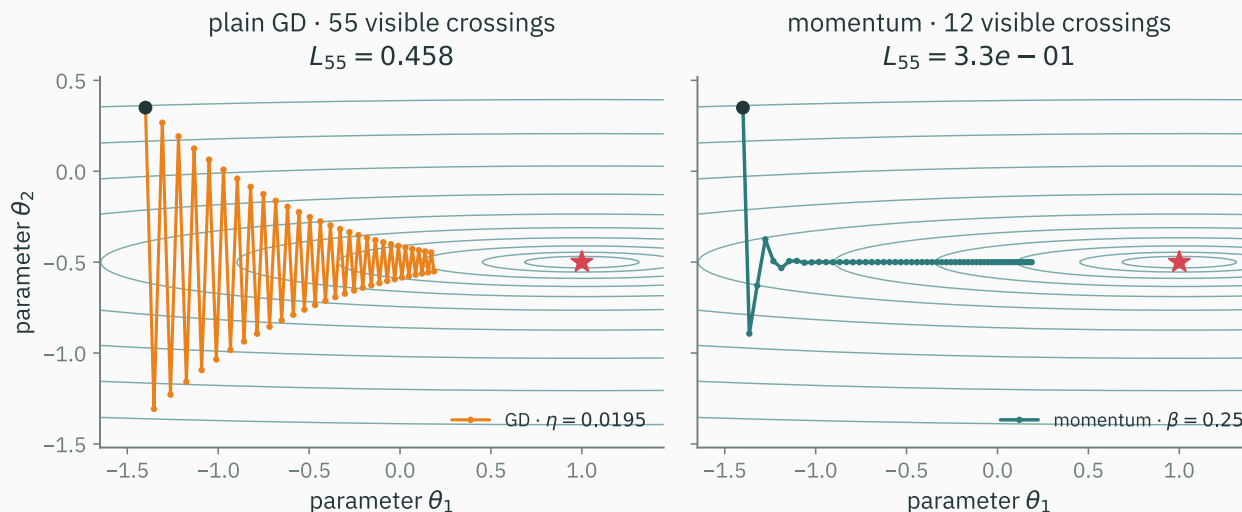
$$+.5750 \uparrow$$

$$-.3108 + .5750 = +.2642 \uparrow$$

$$\text{second coordinate: } -.8931 + .2642 = -.6290$$

The pushes oppose one another, so most of the reversal cancels. Plain GD uses only its current gradient; momentum tempers a reversal using recent history.

Momentum damps the steep-direction oscillation

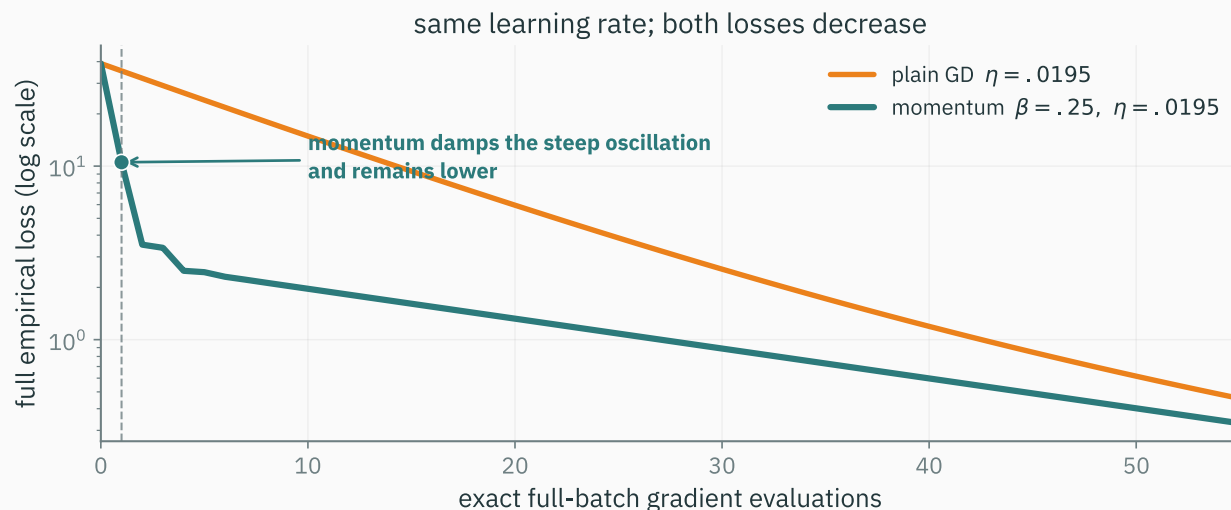


same start and learning rate $\eta = .0195$ · normalized momentum $\beta = .25$ · exact full-batch gradients

COMPUTED · SAME START AND SAME $\eta = .0195$ · 55 EXACT FULL-BATCH GRADIENT EVALUATIONS EACH

Both methods use $\eta = .0195$. Plain GD crosses the valley on every shown update; momentum reduces those crossings while retaining steady motion in θ_1 .

Both losses fall; momentum remains lower



COMPUTED ON EXAMPLE B · IDENTICAL GRADIENT-EVALUATION BUDGET

Both curves use exact full-batch gradients, the same $\eta = .0195$, and both losses decrease at every displayed update. After 55 updates the losses are approximately .330 for momentum and .458 for plain GD.

PLAIN GRADIENT DESCENT

```
def gd_step(theta, grad, eta):  
    return theta - eta * grad
```

Return the updated parameters.

GRADIENT DESCENT WITH MOMENTUM

```
def momentum_step(theta, grad, m, eta, beta):  
    m = beta * m + (1 - beta) * grad  
    theta = theta - eta * m  
    return theta, m
```

Return the parameters and the memory.

These functions assume vectorized arrays or tensors. Compute `grad` at the current `theta`. For momentum, initialize `m = zeros_like(theta)` once and pass the returned *m* into the next update.

PyTorch momentum uses the same training loop

I INTERACTIVE

```
beta, eta = 0.25, 0.0195
opt = torch.optim.SGD(model.parameters(),
                      lr=(1-beta)*eta,
                      momentum=beta)

opt.zero_grad(set_to_none=True)
loss.backward()
opt.step()
```

PyTorch stores the rescaled buffer $\tilde{\mathbf{m}}_{t+1} = \beta\tilde{\mathbf{m}}_t + \hat{\mathbf{g}}_t$. Since $\mathbf{m} = (1 - \beta)\tilde{\mathbf{m}}$, the boxed normalized rule maps to $\text{lr} = (1 - \beta)\eta$. This is the same conceptual η , expressed for PyTorch's scaled state—not a separately tuned rate.

SIGNAL

Compare $\beta =$
.25, .5, .9.

OPTIMIZER

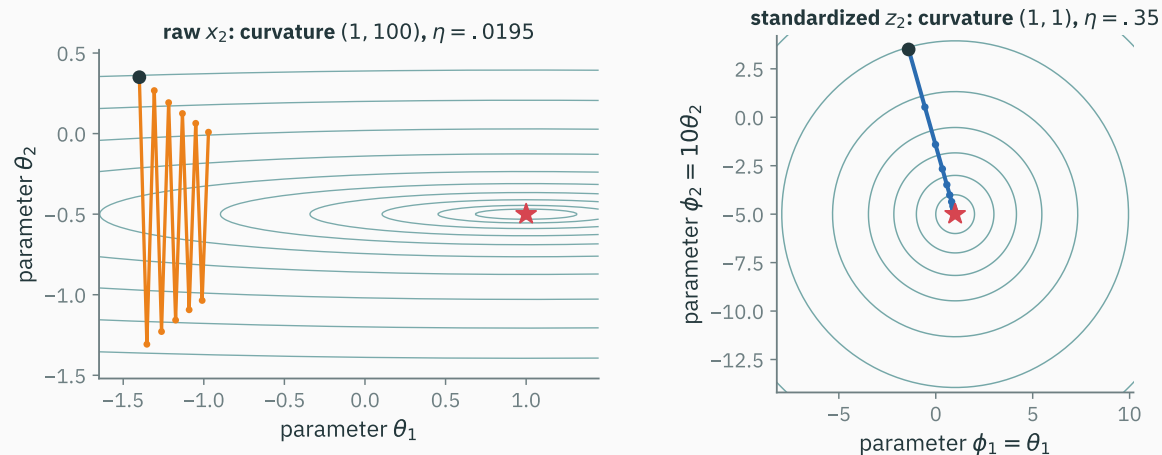
Reproduce the
first six updates.

INTERACTIVE

↗ colab.research.google.com/github/nipunbatra/dl-teaching/blob/master/notebooks/L05/02_ewma_momentum.ipynb

Open **Ravines, EWMA, and momentum** in Colab ↗

Could feature standardization fix this example? Yes.



10 updates each · same starting predictor: $\theta_0 = (-1.4, .35)$ maps to $\phi_0 = (-1.4, 3.5)$

COMPUTED BEFORE/AFTER GEOMETRY FOR $z_2 = \frac{x_2}{10}$

Here standardizing x_2 removes the 100 : 1 scale imbalance. With a stable larger rate, plain GD moves directly toward the optimum—momentum is not needed for this constructed problem.

Four rows: standardized, but still correlated

Use $\hat{y}_i = \theta_1 x_{i,1} + \theta_2 x_{i,2}$, choose $\theta^* = (1, 0)$, and set $y_i = x_{i,1}$:

i	$x_{i,1}$	$x_{i,2}$	y_i
1	-1	$-\frac{7}{5}$	-1
2	-1	$-\frac{1}{5}$	-1
3	1	$\frac{1}{5}$	1
4	1	$\frac{7}{5}$	1

$$\frac{1}{4} \sum_i x_{i,1} = \frac{1}{4} \sum_i x_{i,2} = 0$$

$$\frac{1}{4} \sum_i x_{i,1}^2 = \frac{1}{4} \sum_i x_{i,2}^2 = 1$$

$$\begin{aligned} \text{corr}(x_1, x_2) &= \frac{1}{4} \sum_i x_{i,1} x_{i,2} \\ &= \frac{1}{4} \left(\frac{7}{5} + \frac{1}{5} + \frac{1}{5} + \frac{7}{5} \right) = \frac{4}{5} \end{aligned}$$

Both columns have training-set mean 0 and variance 1, so both are standardized.
Their correlation is nevertheless .8: when x_1 is large, x_2 usually moves with it.

Equal-sized moves change this loss by a factor of nine

Let $e = \theta - \theta^*$. Because $y = X\theta^*$,

$$\mathcal{L}(\theta) = \frac{1}{2N} \|X\theta - y\|_2^2 = \frac{1}{2} e^T \underbrace{\frac{X^T X}{4}}_H e, \quad H = \begin{pmatrix} 1 & \frac{4}{5} \\ \frac{4}{5} & 1 \end{pmatrix}.$$

MOVE TOGETHER · (a, a)

$$\mathcal{L} = \frac{9}{5}a^2$$

Steep direction: the two coefficient changes reinforce.

TRADE OFF · $(a, -a)$

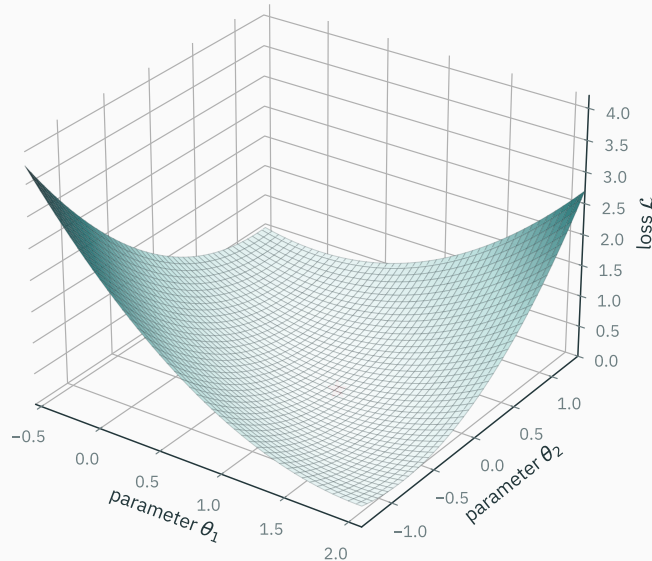
$$\mathcal{L} = \frac{1}{5}a^2$$

Shallow direction: one coefficient compensates for the other.

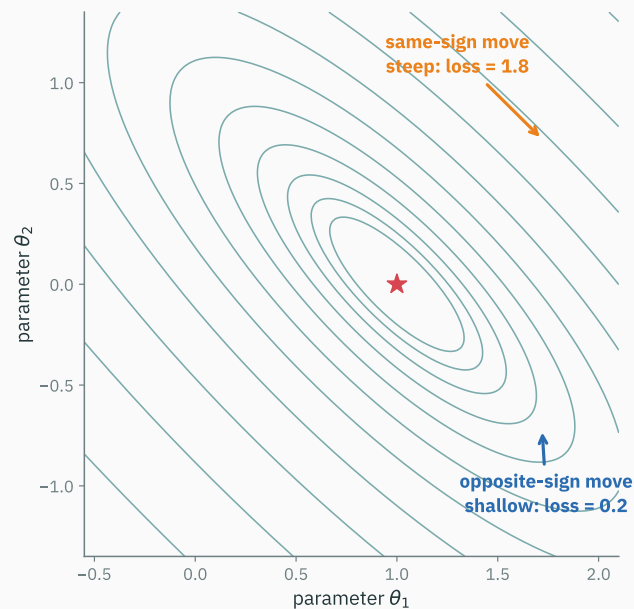
The two moves have the same Euclidean length $\sqrt{2}|a|$, but the first changes the loss nine times as much. That uneven curvature is the ravine.

The 9× curvature difference produces a tilted ravine

The exact four-row loss surface



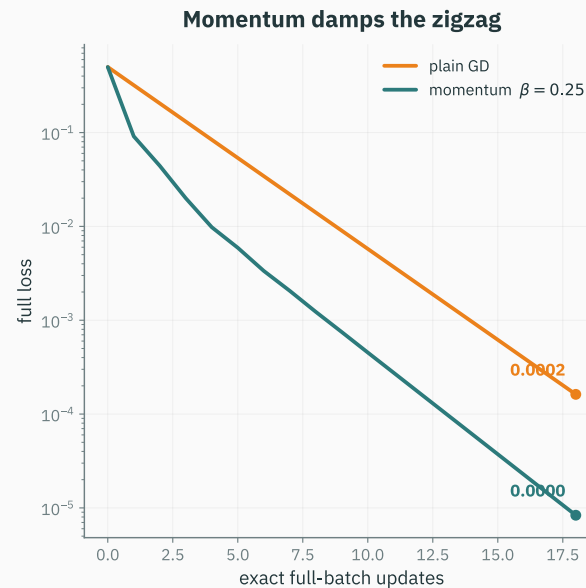
Equal-length moves: 9× different loss



eigenvalues 0.2 and 1.8 · condition number 9 · contour-axis ratio 3

COMPUTED EXACTLY FROM THE FOUR DISPLAYED ROWS · NO SAMPLING NOISE

The same four rows confirm that momentum can help



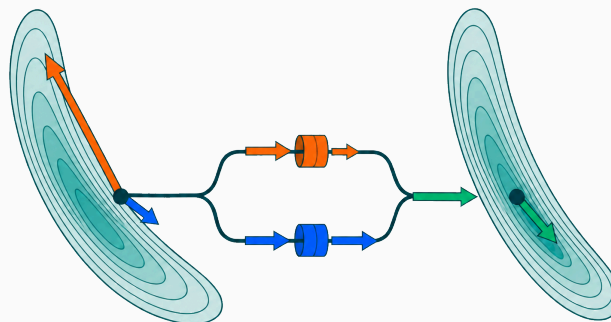
same four rows · same start · same learning rate $\eta = 1$

normalized momentum: $\beta = .25$, $\eta = 1$ · same learning rate as GD · both losses decrease

COMPUTED EXACT FULL-BATCH UPDATES · SAME START AND SAME LEARNING RATE

**A global rate forces a speed–
stability trade-off**

Is this component large relative to its usual size?



CONCEPTUAL ILLUSTRATION · GENERATED WITH OPENAI IMAGEN · NOT DATA

RMSProp averages **squared** gradients, so signs cannot cancel, then divides each component by its own recent RMS scale. A persistently large component gets a larger denominator and no longer forces every coordinate to use a tiny global learning rate.

Example B: recall the data and compute the first gradient

Use $\hat{y}_i = \theta_1 x_{i1} + \theta_2 x_{i2}$. The targets are generated by $\theta^* = (1, -.5)$.

i	x_{i1}	x_{i2}	y_i
1	-1	-10	4
2	-1	10	-6
3	1	-10	6
4	1	10	-4

$$\theta_0 = (-1.4, .35)$$

Averaging the four half-squared errors gives

$$\mathcal{L}(\theta) = \frac{1}{2}(\theta_1 - 1)^2 + 50(\theta_2 + .5)^2$$

$$\nabla_{\theta} \mathcal{L}(\theta) = (\theta_1 - 1, 100(\theta_2 + .5)).$$

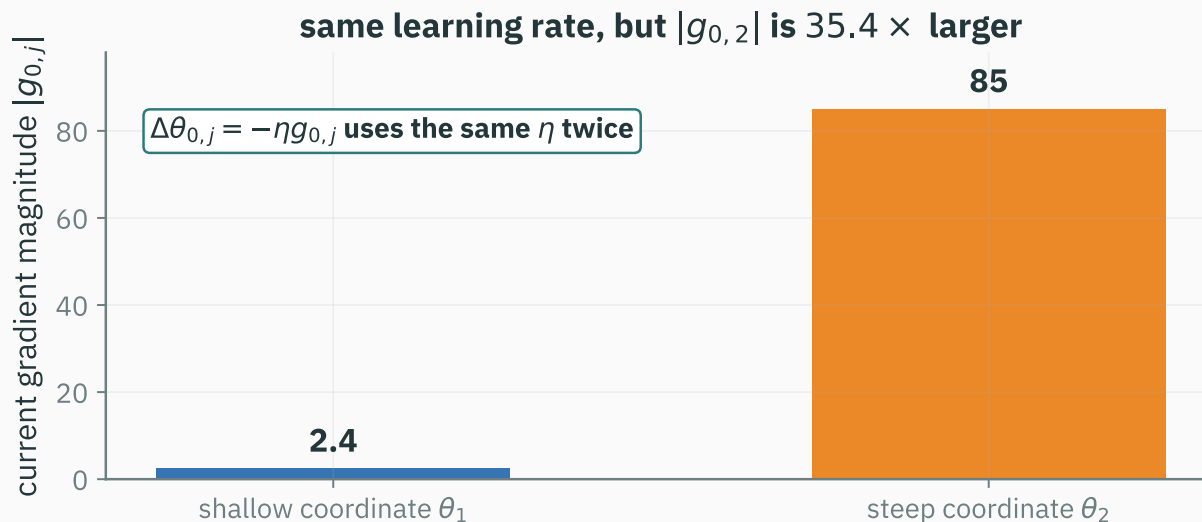
$$g_0 = (-1.4 - 1, 100(.35 + .5)) = (-2.4, 85).$$

$$\frac{|g_{0,2}|}{|g_{0,1}|} = \frac{85}{2.4} \approx 35.4.$$

With $\eta = .0195$, $|\Delta\theta_{0,1}| = .0195(2.4) = .0468$,
 $|\Delta\theta_{0,2}| = .0195(85) = 1.6575$.

$$\frac{1.6575}{.0468} \approx 35.4.$$

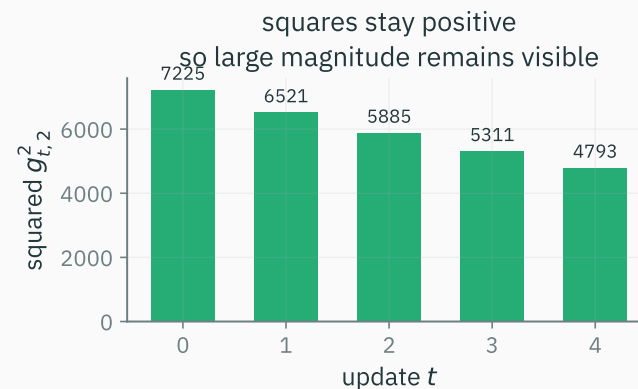
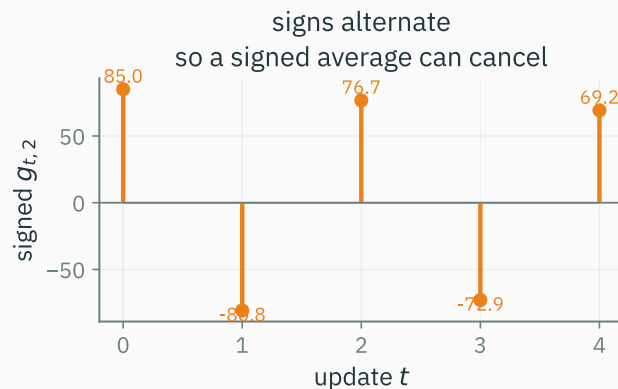
One learning rate scales both gradient components equally



COMPUTED FROM EXAMPLE B $\cdot g_0 = (-2.4, 85)$

The preceding slide calculated these two bars from the four data rows. Multiplying both components by the same η changes their lengths but not their $35.4 \times$ ratio. A small rate is stable but slow in θ_1 ; a larger rate speeds up θ_1 but risks oscillation in θ_2 .

Recent magnitude should control each component's step



computed from the first five exact gradients on the plain-GD ravine path, $\eta = .0195$

COMPUTED FROM THE FIRST FIVE EXACT GRADIENTS ON EXAMPLE B

The sign of the steep component keeps reversing, so an ordinary average can cancel toward zero. Squaring removes the sign and preserves the evidence that this component has been large.

1. **Square:** use $\hat{g}_t^2$ so positive and negative magnitudes do not cancel.
2. **Mean:** call the exponentially weighted mean of those squares v_{t+1} .
3. **Root:** use $r_{t+1} = \sqrt{v_{t+1}}$ to return to gradient units.

v_t is the **mean-square memory**: the running second raw moment of the gradient, with gradient-squared units. The letter v suggests a variance-like magnitude, but this is not a centered variance because we do not subtract the mean. $r_t = \sqrt{v_t}$ is the RMS scale in gradient units.

RMSProp is conventionally expanded as **root mean square propagation**. The RMS scale is the mathematical operation; “Prop” is part of the name, not an extra line in the update.

RMSPROP

$$\hat{\mathbf{g}}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathcal{B}_t}(\boldsymbol{\theta}_t)$$

$$\mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) \hat{\mathbf{g}}_t^2, \quad \mathbf{v}_0 = \mathbf{0}$$

$$\mathbf{r}_{t+1} = \sqrt{\mathbf{v}_{t+1}} + \varepsilon_{\text{opt}}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{g}}_t}{\mathbf{r}_{t+1}}$$

$\mathbf{v}$ stores recent squared-gradient magnitude, and $\mathbf{r} = \sqrt{\mathbf{v}} + \varepsilon_{\text{opt}}$ converts it back to a gradient scale. The next two slides work through the same scalar loss with all three optimizers.

A one-parameter loss we can calculate by hand

$$\mathcal{L}(\theta) = \frac{1}{2}\theta^2, \quad \theta^* = 0.$$

START

$$\theta_0 = 2$$

$$\mathcal{L}(\theta_0) = \frac{1}{2}(2)^2 = 2$$

CURRENT GRADIENT

$$g_0 = \frac{\partial \mathcal{L}(\theta)}{\partial \theta} \bigg|_{\theta=2} = 2$$

All three methods see this same slope.

For the first update, initialize the two memories at zero and use

$$m_0 = 0, \quad v_0 = 0, \quad \eta = .10, \quad \beta = .5, \quad \beta_2 = .9, \quad \varepsilon_{\text{opt}} = 0.$$

The loss, starting point, current gradient, and learning-rate value are the same. Only the quantity transformed before the update is different.

First update on $\mathcal{L}(\theta) = \frac{1}{2}\theta^2$

D DERIVATION

PLAIN GD

$$g_0 = 2$$

$$\Delta\theta_0 = -.10(2) = -.20$$

$$\theta_1 = 2 - .20 = 1.80$$

$$\mathcal{L}_1 = \frac{1}{2}(1.80)^2 = 1.620$$

MOMENTUM

$$m_1 = .5(0) + .5(2) = 1$$

$$\Delta\theta_0 = -.10(1) = -.10$$

$$\theta_1 = 2 - .10 = 1.90$$

$$\mathcal{L}_1 = \frac{1}{2}(1.90)^2 = 1.805$$

RMSPROP

$$v_1 = .9(0) + .1(2^2) = .4$$

$$r_1 = \sqrt{.4} = .632$$

$$\frac{g_0}{r_1} = 2 \div .632 = 3.162$$

$$\Delta\theta_0 = -.10(3.162) = -.316$$

$$\theta_1 = 2 - .316 = 1.684$$

$$\mathcal{L}_1 \approx \frac{1}{2}(1.684)^2 = 1.418$$

GD uses the current gradient. Momentum first forms a signed running average. RMSProp first divides the gradient by its recent root-mean-square scale. This one update checks the formulas; it is not an optimizer ranking.

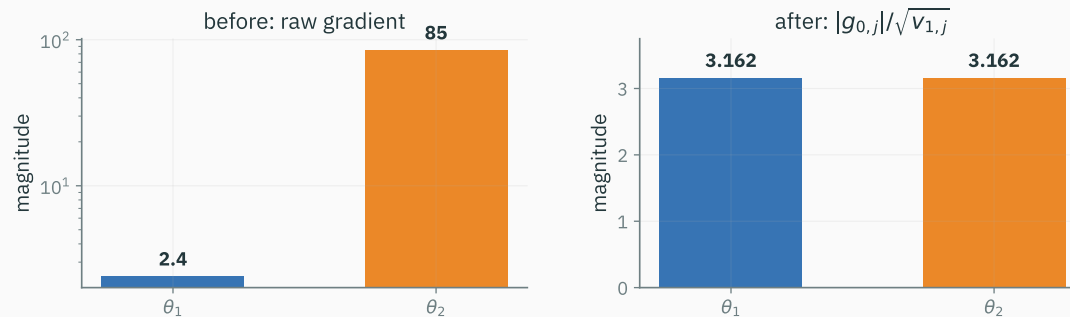
First update: build the two scales

Use $\mathbf{g}_0 = (-2.4, 85)$, $\beta_2 = .9$, and $\mathbf{v}_0 = \mathbf{0}$.

quantity	θ_1 component	θ_2 component
current gradient $\mathbf{g}_0$	-2.4	85
square $\mathbf{g}_0^2$	5.76	7225
$\mathbf{v}_1 = .9\mathbf{v}_0 +$ $.1\mathbf{g}_0^2$	.576	722.5
scale $\mathbf{r}_1 =$ $\sqrt{\mathbf{v}_1}$	.759	26.879

The second component receives the larger denominator because its gradient has been much larger. We omit ϵ_{opt} only in this hand calculation.

First update: divide by the local scales



the 26.879 denominator strongly shrinks the steep coordinate; the 0.759 denominator preserves the shallow one

COMPUTED $\cdot \beta_2 = .9$ EQUAL 3.162 IS A ZERO-START FIRST-UPDATE EFFECT

$$g_0 = (-2.4, 85), \quad r_1 = (.759, 26.879)$$

$$g_0 \div r_1 = (-2.4 \div .759, 85 \div 26.879) = (-3.162, 3.162)$$

$$\Delta\theta_0 = -.08(-3.162, 3.162) = (.253, -.253)$$

$$\theta_1 = (-1.4, .35) + (.253, -.253) = (-1.147, .097)$$

Second update: the history now matters

Carry forward $\theta_1 = (-1.147, .097)$ and $v_1 = (.576, 722.5)$ from update 1.

At θ_1 , remeasure $g_1 = (-2.147, 59.702)$.

quantity	θ_1 component	θ_2 component
$v_2 = .9v_1 + .1g_1^2$	.979	1006.680
$r_2 = \sqrt{v_2}$	.990	31.728
rescaled $\frac{g_1}{r_2}$	-2.170	1.882
$\Delta\theta_1 = -.08\frac{g_1}{r_2}$	.174	-.151

$\theta_2 = (-.973, -.054)$ and $\mathcal{L}_2 = 11.915$. The two rescaled magnitudes are no longer forced to match.

RMSProp adds one persistent state to GD

I INTERACTIVE

PLAIN GD

```
def gd_step(theta, grad, lr):  
    return theta - lr * grad
```

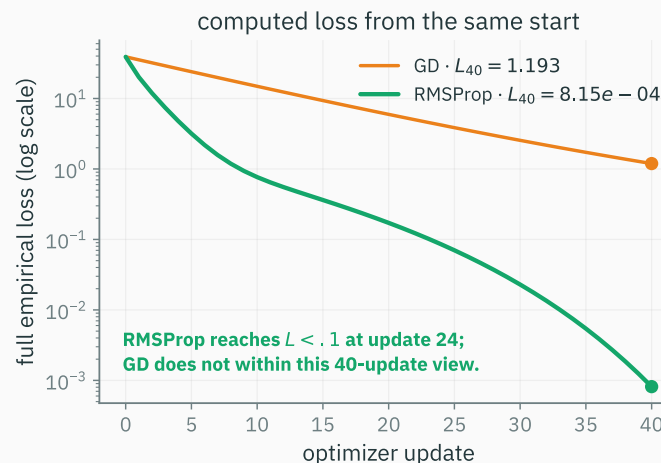
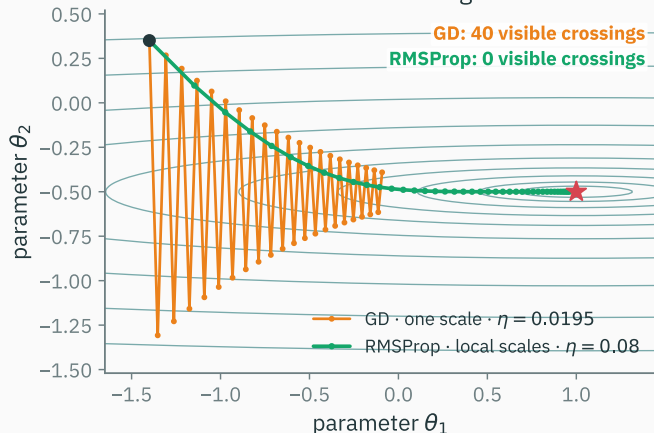
RMSPROP

```
def rmsprop_step(theta, grad, v,  
                  lr, beta2=0.9, eps=1e-8):  
    v = beta2 * v + (1-beta2) * grad.square()  
    theta = theta - lr * grad / (v.sqrt() + eps)  
    return theta, v
```

Initialize $v = \text{zeros_like}(\text{theta})$. PyTorch calls the same decay alpha, so our β_2 is RMSprop(alpha=beta2). The state key square_avg stores v ; the denominator is square_avg.sqrt() + eps.

On Example B, RMSProp removes the visible zigzag

same actual ravine · 40 exact full-batch gradient evaluations each



computed Example B · 40 exact full-batch gradients · GD $\eta = 0.0195$ · RMSProp $\eta = 0.08$, $\beta_2 = 0.9$, $\epsilon = 1e-08$ · method-specific rates, not a benchmark

COMPUTED · SAME OBJECTIVE AND START · 40 EXACT GRADIENTS EACH · METHOD-SPECIFIC RATES SHOWN

The displayed rates are method-specific. Read the path for the rescaling mechanism; Notebook 03 reproduces the first updates, persistent state, and PyTorch parameter path to numerical precision.

Adam combines direction and scale

Momentum stores m_t , a signed-gradient EWMA. Repeated signs reinforce; alternating signs partly cancel.

RMSProp stores v_t , a squared-gradient EWMA. Large recent components receive larger denominators.

When Adam combines the two memories, standalone momentum's β is renamed β_1 for the signed-gradient EWMA. RMSProp's β_2 carries over for the squared-gradient EWMA. The subscripts label the two roles; they do not imply equal numerical values.

What does “moment” mean here?

For a random gradient component G ,

$$\mu'_1 = \mathbb{E}[G] \quad \text{and} \quad \mu'_2 = \mathbb{E}[G^2].$$

The first raw moment is its mean; the second raw moment is its mean square. Adam tracks exponentially weighted, recent versions:

$$\mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1 - \beta_1) \hat{\mathbf{g}}_t$$

$$\mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) \hat{\mathbf{g}}_t^2.$$

Adam stands for **adaptive moment estimation**: it estimates these two moments and uses the second to adapt the scale of the first.

In the zero-start temperature example, observations $1, \dots, t$ contributed weight $1 - \beta^t$. Optimizer measurement $\hat{g}_t$ creates state $t + 1$, so Adam uses $1 - \beta^{t+1}$:

$$\hat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}, \quad \hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}.$$

With $m_0 = v_0 = \mathbf{0}$, $\beta_1 = .9$, and $\beta_2 = .999$, the first update gives

$$m_1 = .1g_0 \rightarrow \hat{m}_1 = g_0, \quad v_1 = .001g_0^2 \rightarrow \hat{v}_1 = g_0^2.$$

Both memories start at zero, and their different β values give different missing masses. Adam normalizes each before combining direction and scale. This corrects initialization shrinkage—not sampling noise or predictive-model bias.

ADAM

$$\mathbf{m}_0 = \mathbf{0}, \quad \mathbf{v}_0 = \mathbf{0}$$

$$\mathbf{m}_{t+1} = \beta_1 \mathbf{m}_t + (1 - \beta_1) \hat{\mathbf{g}}_t$$

$$\mathbf{v}_{t+1} = \beta_2 \mathbf{v}_t + (1 - \beta_2) \hat{\mathbf{g}}_t^2$$

$$\hat{\mathbf{m}}_{t+1} = \frac{\mathbf{m}_{t+1}}{1 - \beta_1^{t+1}}, \quad \hat{\mathbf{v}}_{t+1} = \frac{\mathbf{v}_{t+1}}{1 - \beta_2^{t+1}}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_{t+1}}{\sqrt{\hat{\mathbf{v}}_{t+1} + \epsilon_{\text{opt}}}}$$

TWO RETENTION COEFFICIENTS

$\beta_1 = .9$ retains the signed-gradient memory $\mathbf{m}$.

$\beta_2 = .999$ retains the squared-gradient memory $\mathbf{v}$.

THE SAME VALUES IN PYTORCH

```
opt = torch.optim.Adam(  
    model.parameters(), lr=0.10,  
    betas=(0.9, 0.999), # (beta1, beta2)  
    eps=1e-8,  
)
```


First Adam update on Example B

Use $\beta_1 = .9$, $\beta_2 = .999$, $\eta = .10$, and $g_0 = (-2.4, 85)$.

quantity	θ_1 component	θ_2 component
$m_1 = .1g_0$	-.24	8.5
$v_1 = .001g_0^2$	.00576	7.225
corrected $\hat{m}_1$	-2.4	85
scale $\sqrt{\hat{v}_1}$	2.4	85
step $\Delta\theta_0$	.10	-.10

$\theta_1 \approx (-1.30, .25)$. Ignoring the negligible ε_{opt} , the first Adam step is sign-scaled because the corrected second moment contains only this one gradient.

Second Adam update contains history

At $\theta_1 = (-1.30, .25)$, the new gradient is $g_1 = (-2.3, 75)$.

quantity	θ_1 component	θ_2 component
$m_2 = .9m_1 + .1g_1$	-.446	15.15
$v_2 = .999v_1 + .001g_1^2$	.01104	12.8428
corrected $\hat{m}_2$	-2.347	79.737
scale $\sqrt{\hat{v}_2}$	2.351	80.154
step $\Delta\theta_1$	.0999	-.0995

$\theta_2 = (-1.200, .151)$. From this update onward, both the signed direction and the coordinate scale contain more than one observation.

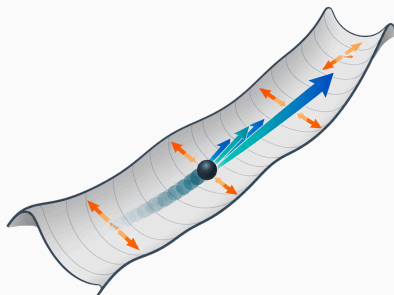
Adam in one vectorized step function

```
def adam_step(theta, grad, m, v, step, lr,
               beta1=0.9, beta2=0.999, eps=1e-8):
    m = beta1 * m + (1-beta1) * grad
    v = beta2 * v + (1-beta2) * grad.square()
    m_hat = m / (1 - beta1**step)
    v_hat = v / (1 - beta2**step)
    theta = theta - lr * m_hat / (v_hat.sqrt() + eps)
    return theta, m, v
```

Initialize `m = zeros_like(theta)`, `v = zeros_like(theta)`, and call the first update with `step=1`. `torch.optim.Adam` stores the same states as `exp_avg` and `exp_avg_sq`.

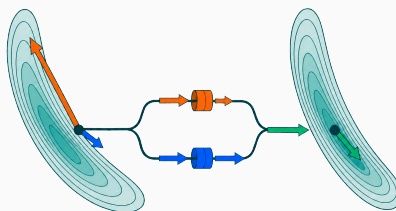
Direction memory, scale memory, or both

MOMENTUM



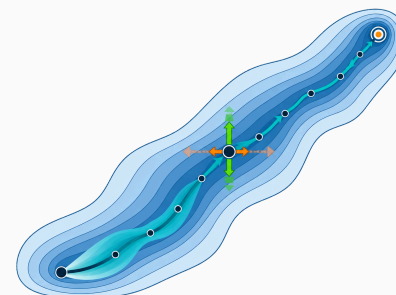
Consistent direction? Average signed gradients.

RMSPROP



Large relative to usual? Divide by the recent RMS scale.

ADAM

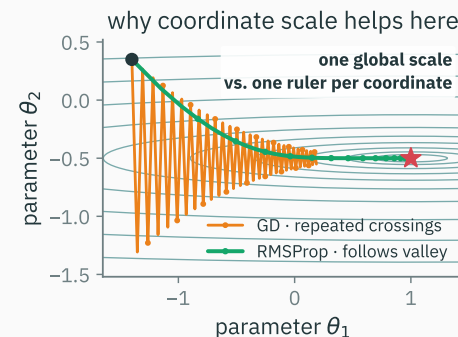
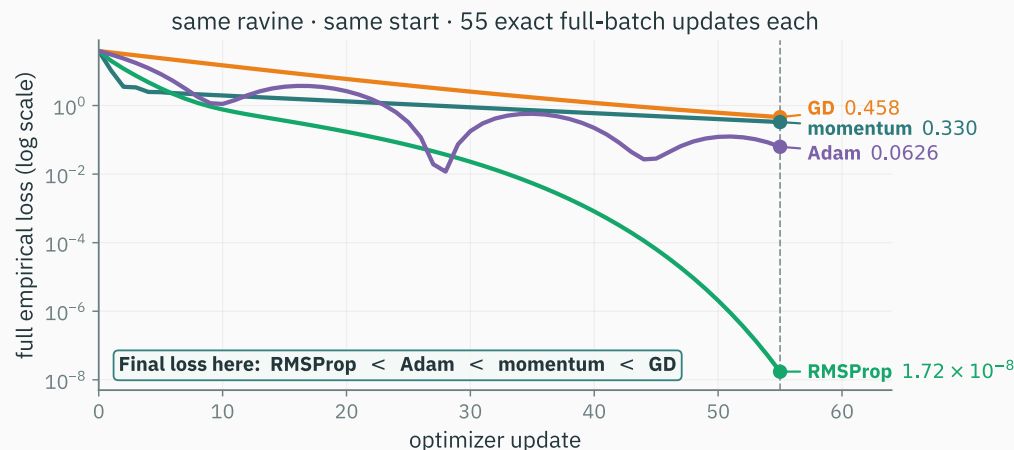


Which direction, at what scale? Combine both memories.

CONCEPTUAL ILLUSTRATIONS • GENERATED WITH OPENAI IMAGEN • NOT DATA

Momentum filters direction over time. RMSProp normalizes each coordinate by its own recent scale. Adam combines the two.

Same ravine, four stored-state mechanisms



GD and normalized momentum share $\eta = .0195$ · momentum $\beta = .25$ · RMSProp $\eta = .08, \beta_2 = .9$ · Adam $\eta = .10, \beta_1 = .9, \beta_2 = .999$
one worked example with method-specific settings · not a universal optimizer ranking

COMPUTED · SAME FULL-BATCH OBJECTIVE · SAME 55-UPDATE BUDGET · METHOD-SPECIFIC RATES SHOWN

On this controlled example, RMSProp and Adam move along the ravine much faster than plain GD. The rates are method-specific and must be tuned again on a different problem.

Notebook 03:

1. recomputes the RMSProp and Adam tables above,
2. runs the same vectorized step functions for complete paths,
3. checks parameters and stored states against PyTorch to numerical precision.

INTERACTIVE

↗ colab.research.google.com/github/nipunbatra/dl-teaching/blob/master/notebooks/L05/03_adaptive_optimizers_constraints.ipynb

Open **Adaptive optimizers, schedules, and constraints** in Colab ↗

Why change the learning rate?

OPTIMIZER DIRECTION AND SCHEDULER SCALE

$$\theta_{t+1} = \theta_t - \eta_t d_t$$

Gradient descent: $d_t = \hat{g}_t$

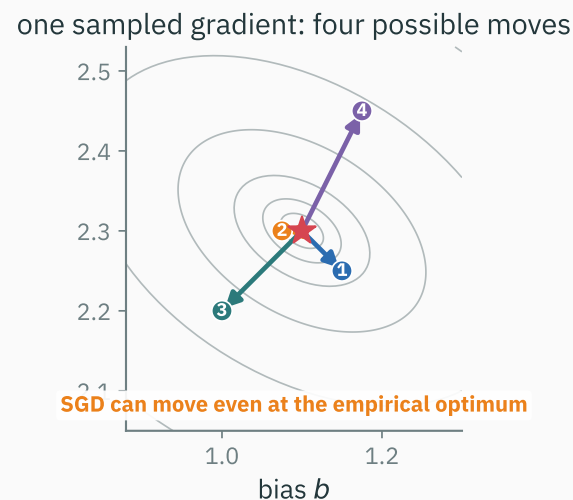
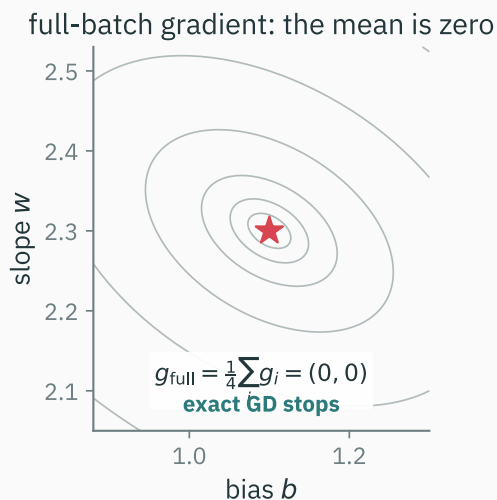
RMSProp: $d_t = \frac{\hat{g}_t}{r_{t+1}}$

Momentum: $d_t = m_{t+1}$

Adam: $d_t = \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1} + \epsilon_{\text{opt}}}}$

The optimizer constructs the direction d_t . The schedule chooses one scalar η_t for that update. Therefore full-batch GD, SGD, momentum, RMSProp, and Adam can all use a schedule.

Why decay is especially useful with stochastic gradients

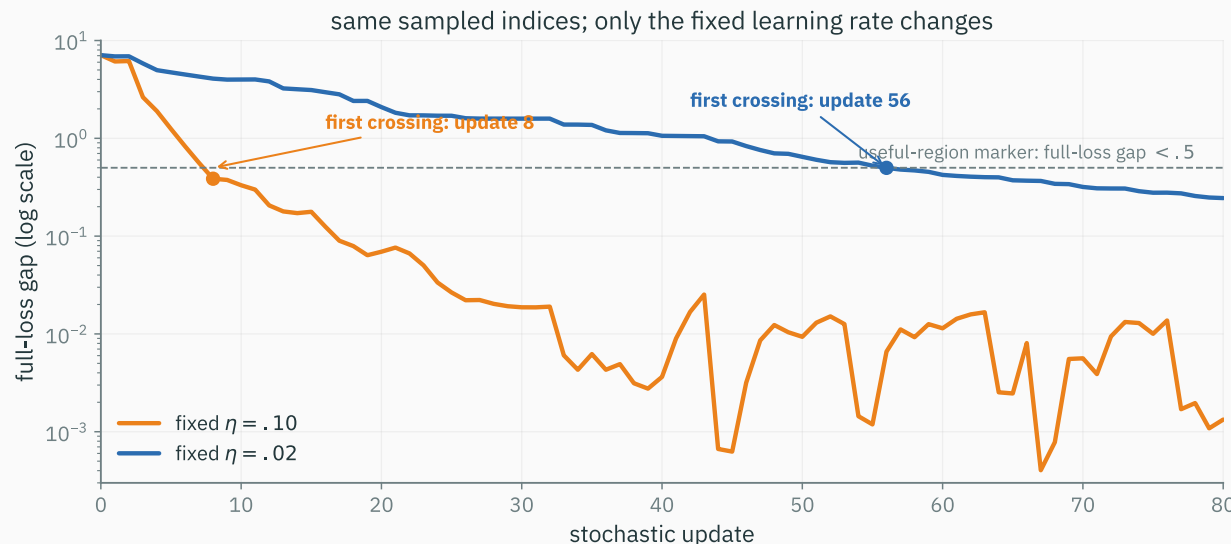


COMPUTED • EXAMPLE A • ALL FOUR SAMPLE GRADIENTS EVALUATED AT THE SAME OPTIMUM

$$\mathbf{g}_1 = (-.2, .2), \quad \mathbf{g}_2 = (.1, 0), \quad \mathbf{g}_3 = (.4, .4), \quad \mathbf{g}_4 = (-.3, -.6).$$

At $\theta^* = (1.1, 2.3)$ their mean is zero, so exact full-batch GD stops. SGD still draws one nonzero arrow. A smaller η_t shortens every such late stochastic move. With exact gradients there is no persistent sampling noise; a well-tuned constant rate may already be enough.

A fixed learning rate forces a choice



COMPUTED • EXAMPLE A • SAME 360 SAMPLED INDICES • SAMPLING WITH REPLACEMENT • SEED 12

With the same sampled-index stream, the fixed rate .10 first crosses a full-loss gap of .5 after 8 updates; the fixed rate .02 needs 56. The larger rate moves quickly early, while the smaller rate is easier to use for fine adjustments later. The marker records a first crossing, not a guarantee that the loss stays below .5.

Inverse-time decay starts large, then shrinks

$$\eta_t = \frac{\eta_0}{1 + dt}, \quad d > 0.$$

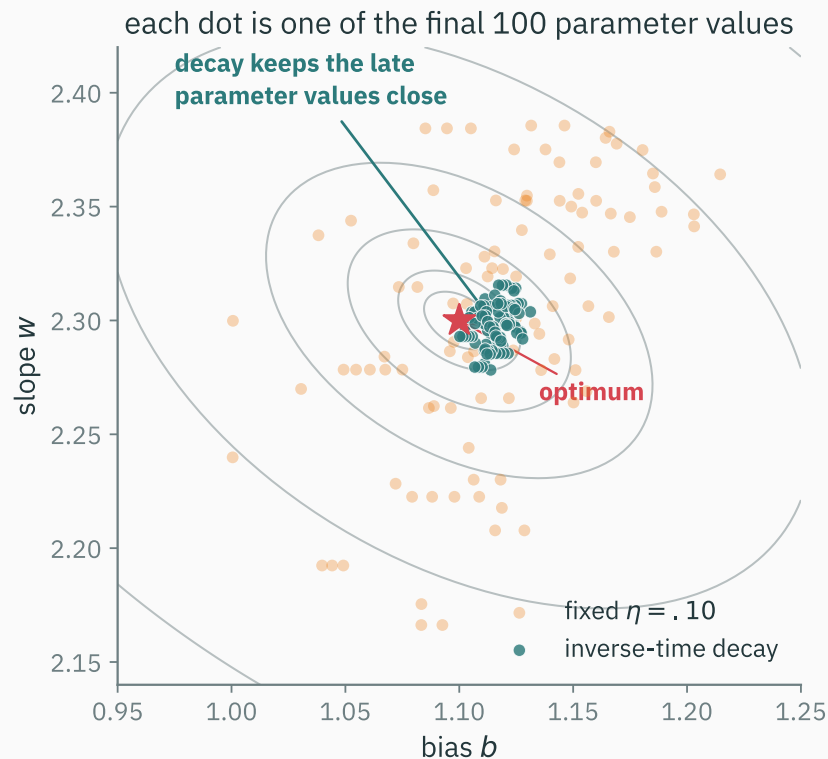
For $\eta_0 = .10$ and $d = .02$:

optimizer update t	rate η_t	interpretation
0	.100	large initial moves
50	.050	half the initial multiplier
350	.0125	smaller late refinements

The intervention is the learning-rate policy. We hold the sampled-index stream and optimizer equations fixed. Because the parameter paths differ, later gradient values and stored-state values generally differ too.

Decay makes the late updates smaller

I INTERACTIVE



Early checkpoint

Both runs start at .10 and first reach a full-loss gap below .5 after 8 updates.

Final 99 moves

3.794 with fixed .10

.520 with decay

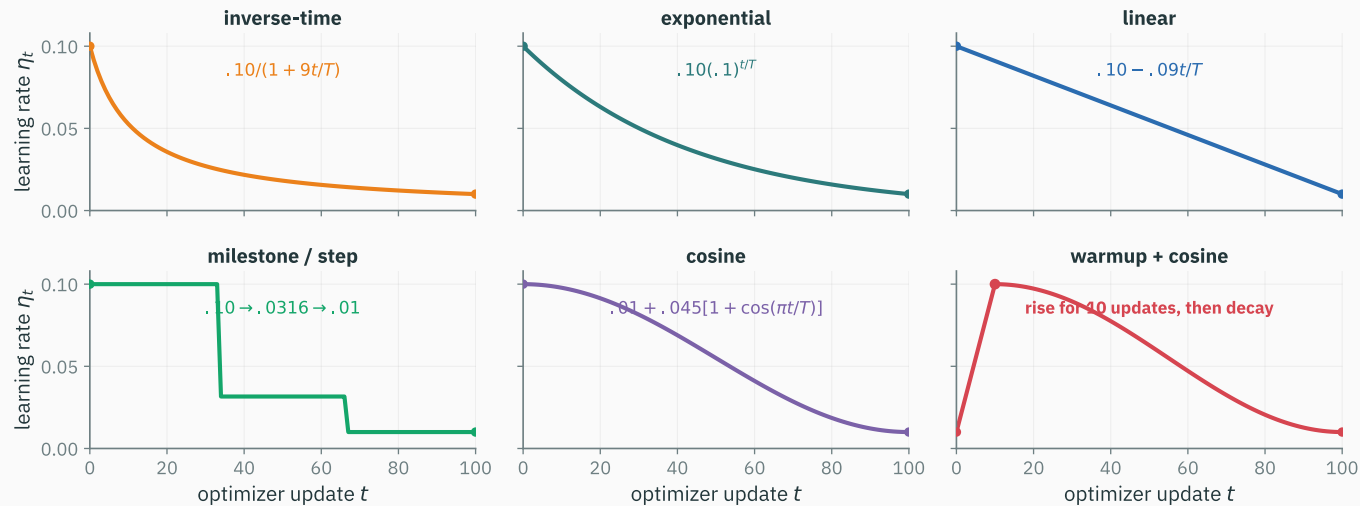
Decay travels about **14% as far**.

COMPUTED • EXAMPLE A • SAME SAMPLED-INDEX STREAM • SEED 12

[Open the Learning-Rate Schedule Explorer ↗](#)

Six common schedule shapes

illustrative schedules over $T = 100$ updates · shapes are not optimizer rankings



COMPUTED SCHEDULE VALUES · ILLUSTRATIVE 100-UPDATE HORIZON

Inverse-time and exponential taper continuously; linear falls at a constant pace; milestones drop abruptly; cosine eases into its floor. Warmup starts below the peak, reaches it gradually, then hands off to decay. These are schedule shapes—not performance rankings.

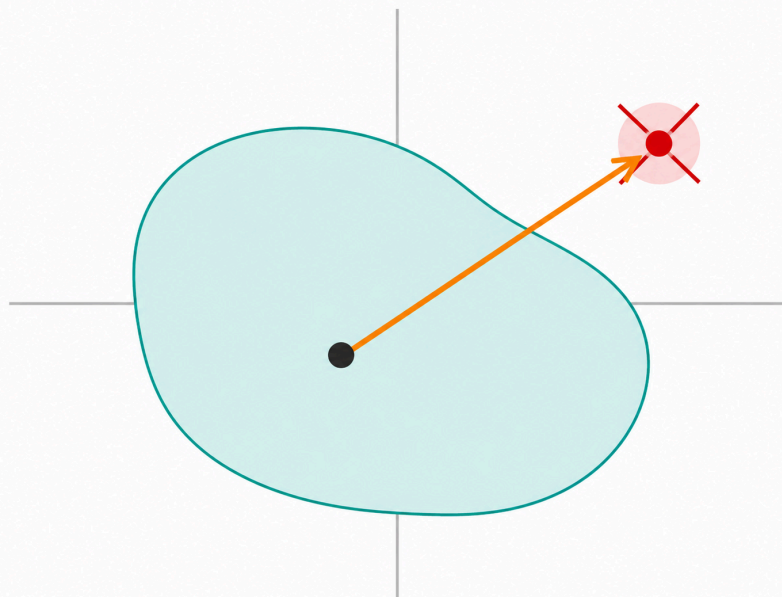
```
epochs = 10
total_updates = epochs * len(loader)
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(
    opt, T_max=total_updates, eta_min=1e-5)

for epoch in range(epochs):
    for xb, yb in loader:
        opt.zero_grad(set_to_none=True)
        batch_loss(model, xb, yb).backward()
        opt.step()          # uses eta_t; changes parameters
        sched.step()        # prepares eta_(t+1)
```

Here the clock is **optimizer updates**. The same scheduler can wrap SGD, momentum, RMSProp, or Adam. With gradient accumulation, advance it after each executed `opt.step()`, not after every microbatch. If an optimizer step is skipped—for example after an AMP overflow—skip the scheduler step too. Epoch-based and validation-based schedulers use different clocks, so name the clock whenever you report one.

Constraints define valid parameter values

An optimizer step can leave the feasible set



CONCEPTUAL ILLUSTRATION · GENERATED WITH OPENAI IMAGEGEN · NOT DATA

1 · Valid start $\theta_t \in \mathcal{C}$: the current parameters satisfy the model's constraint.

2 · Ordinary optimizer step $\tilde{\theta}_{t+1} = \theta_t - \eta \hat{g}_t$ crosses the boundary.

3 · Invalid proposal $\tilde{\theta}_{t+1} \notin \mathcal{C}$, so the model cannot use this value directly.

The feasible set $\mathcal{C}$ is simply the set of all allowed parameter values.

Takeaway: a correct gradient can still propose an invalid parameter. The constraint must be enforced separately; the following slides show two ways.

Why a learned Gaussian scale must be positive

Suppose the network predicts both a mean and an uncertainty. Up to the additive constant $\frac{1}{2} \log(2\pi)$, the per-example negative log-likelihood is

$$y_i \mid x_i \sim \mathcal{N}(\mu_i, \sigma_i^2), \quad \ell_i = \log \sigma_i + \frac{(y_i - \mu_i)^2}{2\sigma_i^2}, \quad \sigma_i > 0.$$

Homoskedastic model

One learned σ is shared by every example. It represents one common noise level for the dataset.

Heteroskedastic model

The network predicts σ_i from x_i . Different inputs may have different noise levels.

At $\sigma_i = 0$, the Gaussian density and the loss are undefined. A negative “standard deviation” has no probabilistic meaning. The constraint is therefore **strict**: $\sigma_i > 0$, not merely $\sigma_i \geq 0$.

One gradient step can make the scale negative

D DERIVATION

For one residual $y - \mu = .05$, start from the valid scale $\sigma_0 = .10$:

1 • MEASURE

$$g_0 = \frac{1}{.1}10 - \frac{.05^2}{.1}10^3 = 7.5$$

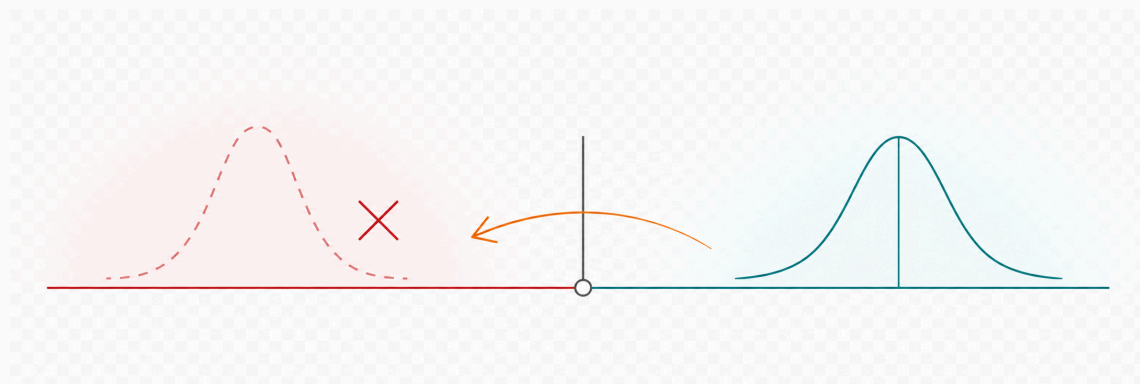
2 • MOVE

$$\Delta\sigma_0 = -.02(7.5) = -.15$$

3 • PROPOSE

$$\tilde{\sigma}_1 = .10 - .15 = -.05$$

| The gradient is correct. The unconstrained update does not encode the requirement $\sigma > 0$.



$$\tilde{\sigma}_1 = -.05 \cdot \text{invalid}$$

the proposal crosses $\sigma = 0$

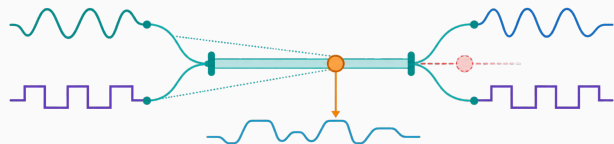
$$\sigma_0 = .10 \cdot \text{valid}$$

CONCEPTUAL ILLUSTRATION • GENERATED WITH OPENAI IMAGEN • VALUES FROM THE WORKED EXAMPLE

A blend coefficient must stay between 0 and 1

Let a model interpolate two predictions:

$$\hat{y} = (1 - c)f_0(x) + cf_1(x), \quad 0 \leq c \leq 1.$$



CONCEPTUAL ILLUSTRATION • GENERATED WITH OPENAI IMAGEN

$c = 0$ uses only f_0 .

$c = .25$ uses 75% of f_0 and 25% of f_1 .

$c = 1$ uses only f_1 .

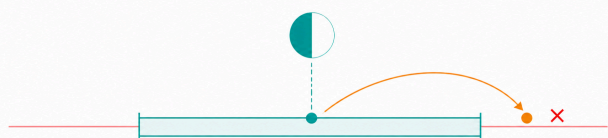
$c = 1.3$ gives weights $(-.3, 1.3)$: this is extrapolation, not the intended blend.

| The allowed set is the closed interval $[0, 1]$: both endpoints are legal and meaningful.

A probability must lie between 0 and 1

For binary classification,

$$y \mid x \sim \text{Bernoulli}(p), \quad p = P(y = 1|x), \quad 0 \leq p \leq 1.$$



CONCEPTUAL ILLUSTRATION · GENERATED WITH OPENAI IMAGEN

$p = .8$ means an 80% chance of class 1.

$p = 1.2$ is impossible: probabilities cannot exceed 1.

The Bernoulli loss is

$$\ell(p, y) = -y \log p - (1 - y) \log(1 - p).$$

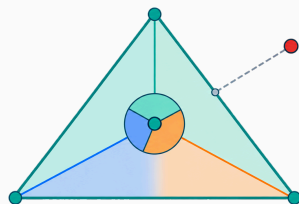
For $p = 1.2$, $\log(1 - p)$ is not real.

We will optimize an unrestricted **logit** and let the sigmoid construct a valid probability inside the model.

Mixture weights must be nonnegative and sum to one

A mixture combines K component distributions:

$$p(y|x) = \sum_{k=1}^K \pi_k p_{k(y|x)}, \quad \pi_k \geq 0, \quad \sum_{k=1}^K \pi_k = 1.$$



CONCEPTUAL ILLUSTRATION · GENERATED WITH OPENAI IMAGEN

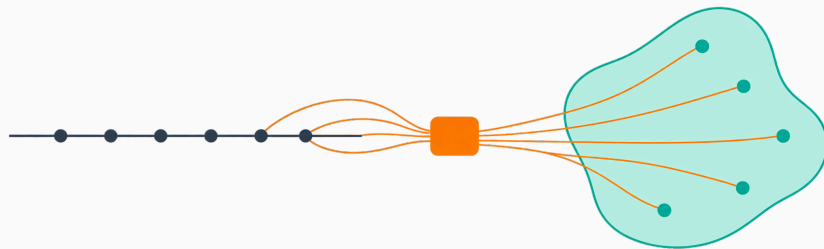
$\pi = (.2, .5, .3)$ is valid: every entry is nonnegative and the total is 1.

$\pi = (.2, .5, .6)$ is invalid: each entry lies in $[0, 1]$, but the total is 1.3.

A valid vector is a point **inside the simplex**.

The coordinates are coupled. Clamping each weight separately into $[0, 1]$ does not guarantee that the weights sum to 1.

Optimize a raw value; transform it before using it



CONCEPTUAL ILLUSTRATION • GENERATED WITH OPENAI IMAGEN

UNCONSTRAINED INPUT



CONSTRAINING FUNCTION



CONSTRAINED OUTPUT

$$\boldsymbol{u} \in \mathbb{R}^d$$

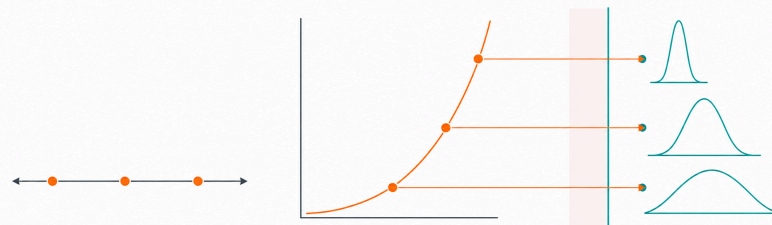
$$\boldsymbol{\theta} = T(\boldsymbol{u})$$

$$\boldsymbol{\theta} \in \mathcal{C}$$

The optimizer changes the unrestricted value $\boldsymbol{u}$. The forward pass applies T before the parameter is used, so the model always sees a valid $\boldsymbol{\theta}$.

Positive scale: real input $\rightarrow$ exponential $\rightarrow$ positive output

D DERIVATION



CONCEPTUAL ILLUSTRATION • GENERATED WITH OPENAI IMAGEN

UNCONSTRAINED INPUT



CONSTRAINING FUNCTION



CONSTRAINED OUTPUT

$$a \in \mathbb{R}$$
$$a = \log(.10) \approx -2.303$$

$$\sigma = \exp(a)$$

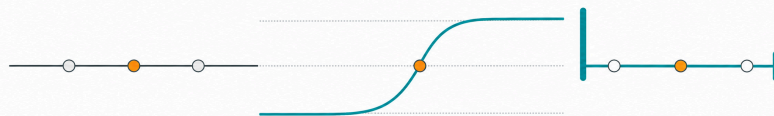
$$\sigma = .10 \in (0, \infty)$$

```
log_sigma = nn.Parameter(torch.tensor(math.log(0.10))) # any real value
sigma = log_sigma.exp()                               # always positive
```

Every finite `log_sigma` produces $\sigma > 0$. For input-dependent noise, the network can output one unrestricted `log_sigma` for each example.

Blend coefficient: real input $\rightarrow$ sigmoid $\rightarrow$ value between 0 and 1

D DERIVATION



CONCEPTUAL ILLUSTRATION · GENERATED WITH OPENAI IMAGEN

UNCONSTRAINED INPUT



CONSTRAINING FUNCTION



CONSTRAINED OUTPUT

$$u \in \mathbb{R}$$
$$u = \log 3 \approx 1.099$$

$$c = \text{sigmoid}(u)$$

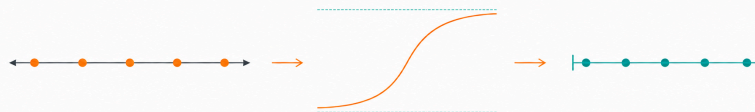
$$c = .75 \in (0, 1)$$

```
raw_c = nn.Parameter(torch.tensor(math.log(3.0))) # unrestricted
c = torch.sigmoid(raw_c)                        # 0 < c < 1
prediction = (1-c) * f0 + c * f1
```

Here $c = .75$, so the blend uses 25% of f_0 and 75% of f_1 . For a general box $a < \theta < b$, use $\theta = a + (b - a)\text{sigmoid}(u)$. Projection is needed if an exact endpoint must be attainable.

Probability: real logit $\rightarrow$ sigmoid $\rightarrow$ valid probability

D DERIVATION



CONCEPTUAL ILLUSTRATION · GENERATED WITH OPENAI IMAGEN

UNCONSTRAINED INPUT



CONSTRAINING FUNCTION



CONSTRAINED OUTPUT

$$z \in \mathbb{R}$$
$$z = \log 4 \approx 1.386$$

$$p = \text{sigmoid}(z)$$

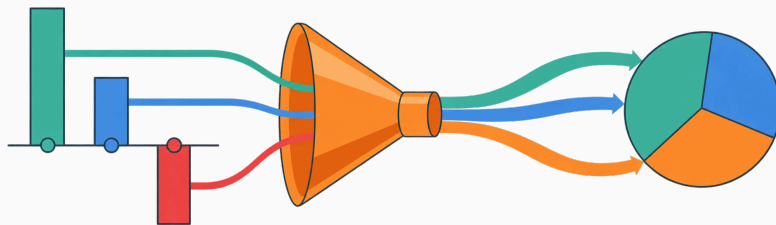
$$p = .8 \in (0, 1)$$

```
logit = model(x) # unrestricted network output
loss = F.binary_cross_entropy_with_logits(logit, y.float())
p = torch.sigmoid(logit) # convert only when a probability is needed
```

Train with logits for numerical stability. The sigmoid guarantees that the reported probability satisfies $0 < p < 1$.

Mixture weights: real logits $\rightarrow$ softmax $\rightarrow$ point in the simplex

D DERIVATION



CONCEPTUAL ILLUSTRATION · GENERATED WITH OPENAI IMAGEN

UNCONSTRAINED INPUT



CONSTRAINING FUNCTION



CONSTRAINED OUTPUT

$$z \in \mathbb{R}^3$$
$$z = (0, \log 2, 0)$$

$$\pi_k = \frac{\exp z_k}{\sum_j \exp z_j}$$

$$\pi = (.25, .50, .25)$$
$$\pi_k > 0, \sum_k \pi_k = 1$$

```
logits = nn.Parameter(torch.tensor([0., math.log(2), 0.])) # unrestricted
pi = torch.softmax(logits, dim=0) # valid weights
```

Softmax transforms all logits together, so positivity and the sum-to-one constraint hold after every forward pass.

Projection repairs a proposal after the optimizer step

First take the ordinary proposal

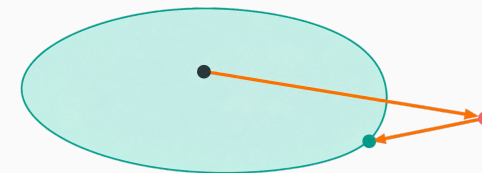
$$\mathbf{u}_{t+1} = \boldsymbol{\theta}_t - \eta \hat{\mathbf{g}}_t.$$

Then return it to the closest allowed point:

$$\boldsymbol{\theta}_{t+1} = \Pi_{\mathcal{C}}(\mathbf{u}_{t+1}).$$

For $\theta \geq 0$: $\Pi_{\mathcal{C}}(u) = \max(0, u)$.

For $a \leq \theta \leq b$: $\Pi_{\mathcal{C}}(u) = \text{clamp}(u, a, b)$.



CONCEPTUAL ILLUSTRATION · GENERATED WITH OPENAI IMAGEN

Projection is natural for a closed feasible set whose boundary is legal. The proposal may leave the set; the parameter used next is valid again.

Project immediately after each optimizer step

I INTERACTIVE

```
c = nn.Parameter(torch.tensor(0.20))
opt = torch.optim.SGD([c], lr=0.40)

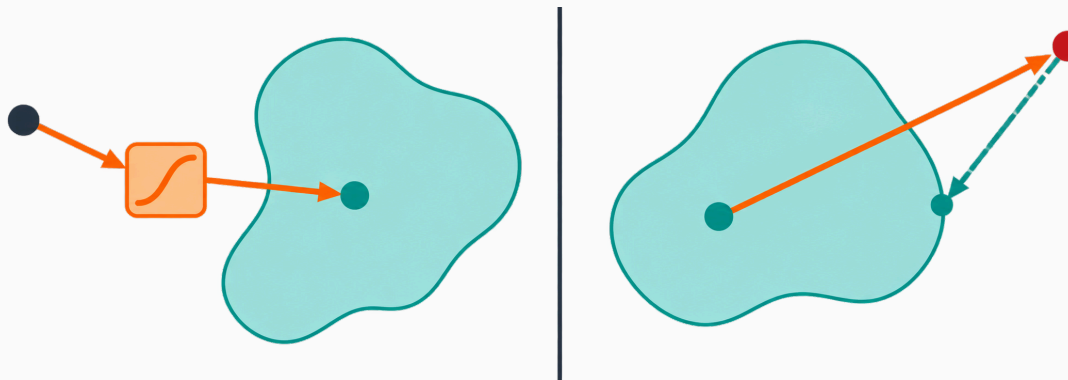
for _ in range(35):
    opt.zero_grad(set_to_none=True)
    loss = 0.5 * (c - 1.4).square()
    loss.backward()
    opt.step()           # propose u
    with torch.no_grad():
        c.clamp_(0.0, 1.0) # project u
```



CONCEPTUAL ILLUSTRATION · GENERATED WITH OPENAI IMAGEN

The unconstrained optimum is 1.4; the box-constrained optimum is the legal boundary $c^* = 1$.

Momentum, RMSProp, and Adam update their stored state before this repair. When projection is frequent, inspect the interaction between that state and the repaired parameter.



CONCEPTUAL ILLUSTRATION • GENERATED WITH OPENAI IMAGEN

Reparameterize. Optimize $r \in \mathbb{R}^d$ and construct $\theta = T(r)$ inside the model. Use this for $\sigma = \exp(a) > 0$, probabilities, and simplex weights.

Project. Make an ordinary proposal, then apply $\Pi_{\mathcal{C}}$. Use this when the legal solution may lie exactly on a closed boundary such as $[a, b]$.

The optimizer supplies the update; the transform or projection enforces the model's constraint.

Choose the repair that matches the symptom

Optimization in one view

EXAMPLES PER GRADIENT

Full batch: exact, slow update. SGD: immediate, noisy. Minibatches balance cost and variance.

DIRECTION MEMORY

Momentum accumulates signed gradients: cancel zigzag and preserve persistent motion.

COORDINATE SCALE

RMSProp rescales using recent squared gradients; Adam adds direction memory.

STEP SIZE OVER TIME

Tune η first. Decay late if updates prevent refinement; state the schedule clock.

VALID PARAMETERS

Reparameterize strict interiors; project only when the boundary is a legal value.

FAIR EVIDENCE

Keep initialization, data order, and update budget fixed; inspect training, validation, and state.

Start simple → diagnose one failure → add one mechanism → measure again.